C_4-Free Subgraphs of the Hypercubes Q_6, Q_7, and Q_8: Odd Squares, Fully Frustrated Models, and Computational Structure

Minamo Minamoto

Independent

minamominamoto4f5683f6@gmail.com

March 2026 (revised August 2026)

This is a revision of a manuscript first circulated as arXiv:2603.29127 (v1–v4, March–May 2026), under the title "New Lower Bounds for C_4-Free Subgraphs of the Hypercubes Q_6, Q_7, and Q_8: Constructions, Structure, and Computational Method." That earlier version announced $\mathrm{ex}(Q_8, C_4) \geq 680$ and did not include the odd-square material of Section 5; see the Acknowledgements for the circumstances of the revision.

Abstract

We study C_4-free subgraphs of the hypercubes Q_6, Q_7, and Q_8, giving explicit lower bounds $\mathrm{ex}(Q_7, C_4) \geq 304$ and $\mathrm{ex}(Q_8, C_4) \geq 682$, together with a computational reproduction of the 132-edge lower-bound construction for $\mathrm{ex}(Q_6, C_4) = 132$, combined with the known upper bound of Harborth and Nienborg [9] (we did not independently verify ILP optimality for the upper bound within a practical runtime; see Section 8). After this manuscript was first circulated, a reader (S. Lai) identified an overlooked connection to a line of statistical-physics literature on fully frustrated hypercubes: under the standard correspondence between edge sets meeting every 4-cycle (*square*) an odd number of times and fully frustrated signed hypercubes, the 1979 construction of Derrida, Pomeau, Toulouse, and Vannimenus [5] already yields a $D \leq 7$ odd-square construction attaining 304 edges at $D = 7$ (we did not reconstruct their specific H_7 edge set), and the D=8 ground state reported by Marinari, Parisi, and Ritort [11] yields a 682-edge C_4-free subgraph of Q_8 that improves our originally announced 680-edge construction. For Q_7, this value (304) is also attained by 389 of our own previously found 304-edge solutions, which we confirm independently satisfy the odd-square condition (none is claimed to be DPTV's specific H_7 construction; we only confirm the attained value, not edge-set identity with theirs). For Q_8, we derived and machine-verified an explicit 682-edge witness attaining the bound MPR report attaining, using their canonical fully frustrated coupling (derivation documented in Section 5.3); this witness is not claimed to be a reconstruction of any particular configuration MPR may have used internally, since they did not publish one. This 682-edge witness is presented here as the paper's headline lower bound in place of the earlier 680-edge witness.

Writing $g(n)$ for the maximum size of an *odd-square* edge set (every square meets it in exactly one or three edges), the numerical values $g(6) = 132$, $g(7) = 304$, and $g(8) = 682$ are, in physics terms, already implicit in the literature: Derrida, Pomeau, Toulouse, and Vannimenus [5] explicitly construct fully-frustrated ground states for $d \leq 7$ (giving $g(6)$ and $g(7)$ under the correspondence below), and Marinari, Parisi, and Ritort [11] report having obtained, by simulated annealing, a $D = 8$ ground state attaining Derrida et al.'s improved energy lower bound (giving $g(8)$ under the same correspondence). The graph-theoretic formulation of the fully-frustrated signed hypercube and its connection to the C_4-free extremal problem is itself not new either: Laplante, Naserasr, Sunny, and Wang [10] explicitly formulate the frustration index of the fully-frustrated signature and its relation to Erdős's C_4-free hypercube question (Section 5.2 below). Our contribution is: (i) for $n = 7, 8$, a self-contained, elementary proof of the upper bounds $g(7) \leq 304$ and $g(8) \leq 682$ (a quadratic field-sum identity together with elementary parity constraints, proved here directly rather than cited from DPTV's Table I, and sharpening LNSW26's general bounds – tight only when n is a power of 4 – to exact values at $n = 6, 7, 8$ specifically); and (ii) explicit, machine-verifiable edge-list certificates for both bounds at $n = 7, 8$, released alongside this paper (Section 5.3). Here the prior literature's constructions are not machine-readable in the same sense: DPTV79 give an explicit recursive scaling construction (their bond configurations for H_5, H_6, H_7, described in prose and figures, not released as a computer-readable edge list), while MPR95 report only an energy value for $D = 8$, without publishing a spin configuration at all. The same holds at $n = 6$, where DPTV79's own $D = 6$ configuration is described in prose; we construct an independent 132-edge odd-square edge list instead, which happens to realise the local-field distribution they report. The exact value $\mathrm{ex}(Q_6, C_4) = 132$ of Harborth and Nienborg [9] is therefore no longer needed to bound $g(6)$ from above, though it remains the source for the unrestricted value. This settles the odd-square subclass completely for $n = 6, 7, 8$ without resolving the unrestricted problem $\mathrm{ex}(Q_n, C_4)$, since a general C_4-free subgraph need not be odd-square.

An exhaustive audit of the 19 866 previously published 304-edge Q_7 solutions shows that only 389 of them lie in the odd-square subclass; the remaining 19 477 do not. The odd-square construction therefore does not subsume or fully explain the structural classification given here (one profile type, Type 18 with 101 solutions, is entirely odd-square; of the other 19, three (ranks 5, 7, and 10) are mixed, containing some odd-square and some non-odd-square solutions, and the remaining 16 contain no odd-square solutions at all): the 19 866 solutions' common structural core (degree sequence $\{4^{32}, 5^{96}\}$, spectral radius in $[4.78543, 4.79129]$, local maximality) and their classification into 20 dimension-profile types are independent results. These 19 866 labelled solutions certainly do *not* exhaust all labelled 304-edge C_4-free subgraphs of Q_7: the first released solution alone has a stabiliser of size 3 in $\mathrm{Aut}(Q_7) \cong (\mathbb{Z}_2)^7 \rtimes S_7$ (order 645 120; exhaustive check over all 645 120 elements), consisting of the identity and two nontrivial elements, both coordinate permutation + XOR pairs – $(5, 1, 2, 3, 4, 6, 0)$ with XOR mask $33 = 0100001_2$, and its square $(6, 1, 2, 3, 4, 0, 5)$ with XOR mask $96 = 1100000_2$, together forming a cyclic group of order 3 – so its orbit alone has size $645\,120/3 = 215\,040 > 19\,866$. What remains open is whether the 19 866 released solutions represent every $\mathrm{Aut}(Q_7)$-orbit of 304-edge C_4-free subgraphs (i.e. every such subgraph is equivalent under $\mathrm{Aut}(Q_7)$ to one of the 19 866), not whether they exhaust the labelled solutions themselves; see Open Problem 3.

For Q_8, the reconstructed 682-edge odd-square witness is locally maximal in a strong sense: all 342 non-edges of Q_8 create at least 3 new C_4s upon addition (compared with the earlier 680-edge simulated-annealing Solution B, for which 2 of 344 non-edges created only 1 or 2 new C_4s; see Section 4 for both 680-edge solutions released). Every C_4-free-construction bound reported here, and every one of the three odd-square values, is witnessed by an explicit edge-list construction with a machine-verifiable certificate. Two distinct 132-edge Q_6 constructions are released and should not be conflated: `q6_edges_132.jsonl` is C_4-free but not odd-square (its square-intersection histogram is $\{1{:}8,\ 2{:}44,\ 3{:}188\}$) and establishes $\mathrm{ex}(Q_6, C_4) \geq 132$, while `q6_odd_square_132.json` is the odd-square witness for $g(6) \geq 132$ (histogram $\{1{:}30,\ 3{:}210\}$). For every bound, a complete, deterministic enumeration of all four-cycles (240 for Q_6, 672 for Q_7, 1 792 for Q_8) establishes C_4-freeness, and every *construction* certificate reported here (the edge sets themselves, their C_4-freeness, and the Q_6 and Q_8 odd-square conditions) is reproducible by an independent, dependency-free verifier (`verify.py`), with the Q_7 odd-square membership of individual solutions separately reproducible via `audit_q7_odd_square.py`. The further structural and statistical claims reported below (spectral-radius ranges, pairwise Hamming extrema, Type-18 automorphism counts, non-edge violation distributions, and similar) were independently recomputed during this paper's preparation but are not packaged into a single bundled verifier; a reader wishing to check them would need to recompute them from the released edge lists using the definitions given in the text.

The Q_7 solutions were found by a two-stage process that we have since traced and partly re-verified (the two stages have different evidentiary status, detailed below and in Section 7): a conventional multi-move-type penalty simulated annealing (add/remove/1-swap/kick moves with temperature-scaled Boltzmann acceptance) found a single seed 304-edge solution, and a remove-and-repair collector – starting from an existing solution (optionally transformed by a random element of $\mathrm{Aut}(Q_7)$), removing a few edges, then greedily re-adding only violation-free edges – mass-collected the released 19 866 solutions; we directly confirmed the collector stage by running it ourselves: from the released seed, in one unseeded 30-second run we observed 1 987 new valid solutions (the script sets no random seed, so this exact count is not expected to reproduce on a rerun; an independent reviewer's rerun found 1 820). Tracing further back, we located most of the prior history, with one correction to an earlier draft of this paper: the earliest recovered attempt (a CBC-solver ILP explicitly commented as attacking Erdős's Problem 100) has a confirmed bug (its C_4-detection finds 0 of 672 four-cycles, since adjacent vertices in the bipartite Q_7 never share a common neighbour, which its logic incorrectly relies on), so we withdraw our earlier claim that this specific recovered file is what reached 289 edges; a recovered solver log, headed as a corrected version and reporting the correct C_4 count, documents that a since-fixed version of the script did reach 289 edges, but we have not located that corrected script itself. From there, file timestamps together with recovered solver logs and a chat transcript document a progression $289 \to 293 \to 299 \to 301 \to 303 \to 304$ edges, the last step apparently via the `sa_search.py` series. We confirmed the 301-edge step's mechanism directly: `source.py`, the HiGHS version, run for 60 seconds with no prior solution to warm-start from (a genuine cold start), produces a valid, growing 295-edge C_4-free solution via branch-and-bound alone. We further ran a format/implementation-consistency check that any reader can reproduce from the released files alone, without needing our recovered archive: applying `sa_collect304.py`'s own canonicalisation-then-MD5 hash function directly to each of the 19 866 released edge sets reproduces the `hash` field already stored in that same released record, for all 19 866 (Section 7); this shows the released hash field and script are mutually consistent, not that this script historically produced these edge sets, which remains a separate, historical-record

claim (recovered logs, chat transcript, timestamps). The two released Q_8 680-edge solutions (Solutions A and B, Section 4) have likewise now been traced to recovered, independently-verified working code, `sa_q8.py`: a penalty-SA hill-climber that always restarts each trial from the current best *valid* (zero-violation) solution rather than from a fresh random sample, progressively improving it (initial greedy construction $\to$ intermediate saved states at $584, 662, \ldots, 679$ edges $\to$ 680). We confirmed the two files this recovered code produced – named `q8_edges_680.txt` and `q8_best.json` in the recovered archive, released here as `q8_solution_a.txt` and `q8_solution_b.json` – match Solutions A and B respectively exactly by edge set, and we confirmed the code makes genuine progress on direct execution: run once under an external 90-second timeout from a saved 670-edge intermediate state, it reduced the violation count at the 675-edge target from 22 to 8. `sa_q8.py` sets no random seed, so this specific numeric trajectory is a one-time observation, not a fixed-seed reproducible result: a reader rerunning the released files should expect qualitatively similar monotonic improvement, not these exact numbers – an independent reviewer's rerun from the same checkpoint observed $25 \to 14$ instead. This is a different script, released separately from `c4free_sa.py`, whose generalisation of the same two-phase idea to a single n-agnostic script (Section 7) does *not* share this always-restart-from-a-valid-state property, and which we found cannot progress from a fresh random start under its own documented parameters for either Q_7 or Q_8; `c4free_sa.py` is retained as released (with its defect documented) but is no longer the operative account of how either the Q_7 or the Q_8 solutions were produced. For Q_6 we additionally give an ILP formulation whose feasible value matches the known exact value $\mathrm{ex}(Q_6, C_4) = 132$; we did not independently verify the ILP's optimality within a practical runtime, and the upper bound itself rests on the combinatorial proof of Harborth and Nienborg [9], which we reproduce on the lower (construction) side.

Edge lists, ILP files, the odd-square reconstruction and audit scripts, and source code are made available at
https://github.com/minamominamoto/c4free-hypercube.

MSC 2020: 05C35, 05C62, 05C50.
Keywords: hypercube, C_4-free, extremal graph theory, simulated annealing, integer linear programming, dimension profile.

Contents

1 Introduction

The n-dimensional hypercube Q_n has vertex set $\{0,1\}^n$, two vertices adjacent iff they differ in exactly one coordinate; it has 2^n vertices and $n \cdot 2^{n-1}$ edges. The problem of determining $\mathrm{ex}(Q_n, C_4) = \max\{|E(G)| : G \subseteq Q_n,\ G \text{ is } C_4\text{-free}\}$ was raised by Erdős [8] and remains open in general.

The best known asymptotic bounds are

$$\tfrac{1}{2}(n+\sqrt{n}) \cdot 2^{n-1} \;\leq\; \mathrm{ex}(Q_n, C_4) \;\leq\; (0.60318 + o(1)) \cdot n \cdot 2^{n-1}, \tag{1}$$

where the lower bound (valid for $n = 4^r$) is due to Brass, Harborth, and Nienborg [3]. The upper bound is a statement about the edge Turán *density* $\pi_e = \limsup_n \mathrm{ex}(Q_n, C_4)/(n2^{n-1})$, not a pointwise inequality valid at every finite n; the constant 0.60318 is due to Baber [1], who used the semidefinite flag algebra method to improve the earlier bound of Thomason and Wagner [12]; the intermediate value 0.6068 was obtained independently by Baber [1] and by Balogh, Hu, Lidický, and Liu [2]. For general $n \geq 9$, the weaker bound $\frac{1}{2}(n + 0.9\sqrt{n}) \cdot 2^{n-1}$ applies [3]. Exact values have been determined for small n; Harborth and Nienborg [9] proved $\mathrm{ex}(Q_6, C_4) = 132$.

Our main results are:

Theorem 1. *(i)* $\mathrm{ex}(Q_7, C_4) \geq 304$.

(ii) $\mathrm{ex}(Q_8, C_4) \geq 682$.

Part (i) is not new: it follows from the 1979 construction of Derrida, Pomeau, Toulouse, and Vannimenus [5] (Section 5), and we present it here as an independently reproduced and machine-verified result rather than as a novel bound. Part (ii) improves our originally announced bound $\mathrm{ex}(Q_8, C_4) \geq 680$, but here too the bare existence claim is not new in substance: Marinari, Parisi, and Ritort [11] already report having exhibited a $D = 8$ ground state attaining Derrida et al.'s improved energy lower bound, and combined with the signed-graph/odd-square correspondence and edge-count–field-sum translation we give in Section 5, their published existence claim already implies the existence of a 682-edge odd-square subgraph of Q_8, independently of any witness search of our own. What Part (ii) contributes beyond that prior existence claim is: an explicit graph-theoretic translation of MPR's result into edge-count terms (Section 5.2); a publicly released, explicit, machine-verifiable 682-edge certificate, which MPR's paper does not supply (they report the result in energy units only, without an explicit spin configuration); and the structural analysis of that certificate (Proposition 13). The specific certificate we release attains the same bound value MPR report; its own provenance is recorded as a dated private-correspondence account rather than an independently reproducible computational log (see Section 5.3); we do not claim it reproduces any particular configuration MPR may have used internally. Both bounds are realised by explicit constructions (supplementary files `q7_edges_304.jsonl.part1--3` and `q8_odd_square_682.json`; the originally announced 680-edge Q_8 witness, `q8_edges_680.jsonl`, is retained as a simulated-annealing data point, see Section 4).

Within the *odd-square* subclass — edge sets meeting every square in an odd number of edges — the extremal value $g(n)$ is determined exactly for $n = 6, 7, 8$:

Theorem 2. *$g(6) = 132$, $g(7) = 304$, and $g(8) = 682$.*

All three values are self-contained within the released materials: each upper bound follows from Lemma 9 and Lemma 11 (Section 5.2), and each lower bound is witnessed by an explicit, machine-verified odd-square edge list released with this paper (Section 5.3). The citations to [5, 11] record historical priority for the values, not a dependency of the proofs. One point of possible confusion: the released 132-edge Q_6 construction `q6_edges_132.jsonl` is a genuine, independently machine-verified C_4-free witness for $\mathrm{ex}(Q_6, C_4) \geq 132$, but it is *not* odd-square; the odd-square witness for $g(6) \geq 132$ is the separate file `q6_odd_square_132.json`.

Theorem 2 is proved in Section 5. Combined with $g(n) \leq \mathrm{ex}(Q_n, C_4)$, it in fact implies Theorem 1's lower bounds $\mathrm{ex}(Q_7, C_4) \geq 304$ and $\mathrm{ex}(Q_8, C_4) \geq 682$ directly (redundantly with the explicit constructions of Sections 3–4). What it does *not* resolve is whether these bounds are exact, i.e. whether $\mathrm{ex}(Q_7, C_4) = 304$ and $\mathrm{ex}(Q_8, C_4) = 682$: a general C_4-free subgraph need not be odd-square, so $g(n) = \mathrm{ex}(Q_n, C_4)$ does not follow automatically, and of the 19 866 published 304-edge Q_7 solutions, only 389 are odd-square (Section 6).

The paper is organised as follows. Section 2 describes the C_4 enumeration and verification procedure used throughout. Sections 3–4 present the originally announced Q_7 and Q_8 simulated-annealing constructions. Section 5 introduces the odd-square subclass, proves Theorem 2, and presents the reconstructed 682-edge Q_8 witness and its verification. Section 6 classifies the 19 866 Q_7 solutions found by their dimension profiles, yielding exactly 20 types, and reports the odd-square audit (389 of 19 866). Section 7 describes the two-phase simulated annealing algorithm. Section 8 reproduces $\mathrm{ex}(Q_6, C_4) = 132$ computationally on the lower-bound side, with an ILP formulation for the upper-bound side. Section 9 lists open problems.

2 C_4 Enumeration and Verification

Lemma 3 (C_4 enumeration in Q_n)**.** *Every four-cycle of Q_n is uniquely determined by a pair of coordinate directions $0 \leq i < j \leq n-1$ and a base vertex $b \in \{0,1\}^n$ with $b_i = b_j = 0$. Its four vertices are*

$$b, \quad b \oplus e_i, \quad b \oplus e_j, \quad b \oplus e_i \oplus e_j,$$

Table 1: Known values and lower bounds of $\mathrm{ex}(Q_n, C_4)$. The BHN column gives Brass–Harborth–Nienborg estimates from Eq. (1); that equation's tight form applies only for $n = 4^r$ and its weaker form only for $n \geq 9$ [3], so the $n = 5, 6$ entries are not covered by either stated form (marked n/a) and the $n = 7, 8$ entries are extrapolations of the $n \geq 9$ form *beyond* its stated domain, included for illustrative comparison only, not as a proven bound. In the Source column, "value; witness ours" means the numerical value has historical priority in the cited work, which published no machine-readable edge list or spin configuration for it; the explicit certificate released here is the present author's own and is not claimed to reproduce any configuration of the cited work.

n	$\lvert E(Q_n)\rvert$	$\mathrm{ex}(Q_n, C_4)$	BHN (Eq. 1)	Source
4	32	24	24 (tight, $n = 4^1$)	Chung [4]
5	80	56	n/a (outside domain)	Emamy-K et al. [6]
6	192	132	n/a (outside domain)	Harborth–Nienborg [9]
7	448	$\geq$ **304**	≈ 300.2 (extrapol.)	value [5]; witness ours
8	1024	$\geq$ **682**	≈ 674.9 (extrapol.)	value [11]; witness ours

and the total number of four-cycles in Q_n is $\binom{n}{2} \cdot 2^{n-2}$. For $n = 6$: 240; for $n = 7$: 672; for $n = 8$: 1 792.

Verification algorithm. Given a candidate edge set $E \subseteq E(Q_n)$ stored as a hash set, the following procedure performs a complete, deterministic C_4-freeness check:

> **for each** pair (i, j) with $0 \leq i < j \leq n - 1$:
> **for each** $b \in \{0, 1\}^n$ with $b_i = b_j = 0$:
> let $v_1 = b$, $v_2 = b \oplus e_i$, $v_3 = b \oplus e_j$, $v_4 = b \oplus e_i \oplus e_j$
> **if** $\{v_1v_2, v_1v_3, v_2v_4, v_3v_4\} \subseteq E$: **return** VIOLATION
> **return** C4-FREE

We applied this algorithm to all constructions and all 19 866 solutions of Q_7; every call returned C4-FREE.

3 The Q_7 Construction

We constructed a C_4-free subgraph of Q_7 on 304 edges by penalty-based simulated annealing (Section 7, where the production code has since been recovered and verified), then certified it by Lemma 3's algorithm.

Proposition 4. *The specific 304-edge seed construction just described (the first solution found, not a generic member of the later 19 866-solution ensemble of Section 6) has the following properties:*

(i) Degree sequence: 32 vertices of degree 4 and 96 of degree 5 (degree sum $608 = 2 \cdot 304$).

(ii) Spectral radius $\lambda_1 \approx 4.787$ (a genuine computed quantity for this construction); $\lambda_n \approx -4.787$, i.e. $\lambda_1 + \lambda_n = 0$ to machine precision, which is automatic for any subgraph of the bipartite Q_7 and serves here only as a sanity check on the eigenvalue computation, not as an independent structural finding.

(iii) $\mathrm{Tr}(A^4) = 5216$, *attaining the C_4-free trace equality* $\sum_i \deg(i)^2 + \sum_{uv\in E}(\deg u + \deg v) - 2|E|$.

(iv) Local maximality: every non-edge of Q_7 creates a C_4 when added.

Across the collector runs described in Section 7 (whose `trial` field reaches into the millions per run; 19 866 is the number of *stored solutions*, not the number of trials), we obtained 19 866 C_4-free subgraphs of Q_7 on 304 edges with pairwise distinct edge sets (not quotiented by $\mathrm{Aut}(Q_7)$), all sharing the structural properties stated in Proposition 16.

4 The Q_8 Construction

This section documents the originally announced Q_8 witness, found by a penalty-SA hill-climber distinct from the Q_7 construction's method (see Section 7 for the recovered and verified `sa_q8.py` production code). It has since been superseded as the paper's headline lower bound by the 682-edge odd-square reconstruction of Section 5; it is retained here as an independent structural data point.

Simulated annealing produced the two C_4-free subgraphs of Q_8 on 680 edges (both released in `q8_edges_680.jsonl`), each certified by exhaustive enumeration of all 1 792 four-cycles. We refer to them as Solution A (first line of the file) and Solution B (second line); the two are structurally distinct and are kept separate below, since an earlier draft of this section inadvertently described properties of Solution A and Solution B as if they belonged to a single construction.

Proposition 5. *Both 680-edge solutions are C_4-free and locally maximal (every non-edge of Q_8 creates at least one C_4 upon addition), with edge density $680/1024 = 66.4\%$ (compared with $304/448 = 67.9\%$ for the Q_7 construction) and $\lambda_1 + \lambda_n = 0$ (automatic for any subgraph of the bipartite Q_8; not by itself a distinguishing structural feature). They differ as follows:*

(i) Solution A: degree sequence $\{4^8, 5^{160}, 6^{88}\}$ *(degree sum* $1360 = 2\cdot 680$*); square-intersection histogram* $\{1:308,\ 3:1484\}$ *over the* 1 792 *squares (incidentally odd-square, though not constructed as such – see below for the implication of this); non-edge violation distribution* $\{2:3,\ 3:48,\ 4:144,\ 5:136,\ 6:13\}$ *(minimum* 2 *new* C_4*s per non-edge).*

(ii) Solution B: degree sequence $\{4^7, 5^{162}, 6^{87}\}$ *(degree sum* $1360 = 2\cdot 680$*); square-intersection histogram* $\{1:302,\ 2:12,\ 3:1478\}$ *(not odd-square:* 12 *squares meet it in exactly* 2 *edges); non-edge violation distribution* $\{1:1,\ 2:1,\ 3:49,\ 4:153,\ 5:124,\ 6:16\}$ *(minimum* 1 *new* C_4*, attained by exactly one non-edge).*

Solution A's odd-squareness has a notable implication for the timeline of results in this paper: since it was originally announced (before the 682-edge odd-square reconstruction of Section 5 existed) and is itself odd-square, the bound $g(8) \geq 680$ was already achieved within the odd-square subclass at that earlier point, not only via the later, deliberately-constructed 682-edge witness. We did not notice or exploit this at the time; it is recorded here as a fact about the released data, confirmed independently by direct square-by-square enumeration.

Remark 6. Brass–Harborth–Nienborg's tight bound $\frac{1}{2}(n + \sqrt{n}) \cdot 2^{n-1}$ applies only for $n = 4^r$, and their weaker bound $\frac{1}{2}(n + 0.9\sqrt{n}) \cdot 2^{n-1}$ is stated only for $n \geq 9$ (Eq. (1), [3]); neither covers $n = 8$. Evaluating the $n \geq 9$ form at $n = 8$ nonetheless gives ≈ 674.9; both 680-edge constructions and the 682-edge odd-square reconstruction of Section 5 exceed this extrapolated figure (by 5.1 and 7.1 edges respectively), but we do not claim this as a comparison against a proven bound.

4.1 Local optimality of the 680-edge constructions

Solution B is locally maximal in the weakest possible sense among the two: among its 344 non-edges, exactly 1 creates exactly 1 new C_4; exactly 1 creates exactly 2; the remaining 342 create 3 or more (Proposition 5(ii)). Solution A's non-edges all create at least 2 new C_4s.

For historical completeness only, not as a reviewable computational result: the manuscript that originally announced the 680-edge construction also reported a failed search for 681 edges,[1] which we mention only as development history, not as evidence bearing on any current claim in this paper.

Table 2 compares the Q_7 construction with Solution B (the 680-edge solution whose local-optimality numbers are discussed above); see Table 3 in Section 5 for the odd-square reconstructions.

Table 2: Structural comparison of the Q_7 construction and Q_8 Solution B (see Proposition 5 for Solution A).

Parameter	Q_7 (304 edges)	Q_8 Solution B (680 edges)
Total edges $\lvert E(Q_n)\rvert$	448	1024
Achieved edges	304	680
Edge density	67.9%	66.4%
Min degree	4	4
Max degree	5	6
Degree sequence	$\{4^{32}, 5^{96}\}$	$\{4^7, 5^{162}, 6^{87}\}$
BHN (Eq. 1), extrapolated	≈ 300.2	≈ 674.9
Improvement	+3.8	+5.1

[1] Over 1 076 reported simulated-annealing trials targeting 681 edges, the minimum C_4 violation count observed was 1 (never 0). This was cited at the time as computational evidence for the conjecture $\mathrm{ex}(Q_8, C_4) = 680$; that conjecture is withdrawn, since the odd-square reconstruction of Section 5 gives $\mathrm{ex}(Q_8, C_4) \geq 682 > 680$, so the figure carries no remaining extremal significance. Beyond that withdrawal, given Section 7's finding that the released, unmodified heuristic cannot reliably progress from a fresh random start under its documented parameters, this statistic should not be read as an experimental result about the behaviour of the *stated* (i.e. released, as-documented) heuristic at all – it is an unarchived, author-recorded historical claim whose relationship to the released code we cannot verify, with no seed list or log supplied. A stochastic annealing run does not in any case define a graph-theoretic neighbourhood search, so even taken at face value this figure would be evidence only about one particular unreproduced search process.

5 The Odd-Square Subclass

5.1 Definition and equivalence to fully frustrated hypercubes

Definition 7. An edge set $E \subseteq E(Q_n)$ is *odd-square* if every square (four-cycle) of Q_n meets E in an odd number of edges, i.e. one or three of its four edges lie in E. Write $g(n) = \max\{|E| : E \text{ is odd-square in } Q_n\}$.

Every odd-square edge set is C_4-free, since no square can meet it in all four edges; hence $g(n) \le \mathrm{ex}(Q_n, C_4)$. The odd-square condition is exactly the condition studied under a different name in the statistical-physics literature on *fully frustrated hypercubes*: assign to each edge e a sign $\sigma(e) = +1$ if $e \in E$ and $\sigma(e) = -1$ otherwise; the square condition becomes "the product of signs around every square is -1", i.e. every square is frustrated.

Lemma 8. *Let $\sigma_1, \sigma_2 : E(Q_n) \to \{\pm 1\}$ both have product -1 on every square. Then there is a function $t : \{0,1\}^n \to \{\pm 1\}$ with $\sigma_2(x,y) = \sigma_1(x,y)\, t(x)t(y)$ for every edge (x,y).*

Proof. Let $\tau = \sigma_1\sigma_2$ (pointwise product). On every square C,

$$\prod_{e\in C} \tau(e) = \Big(\prod_{e\in C} \sigma_1(e)\Big)\Big(\prod_{e\in C} \sigma_2(e)\Big) = (-1)(-1) = +1.$$

The map sending a cycle C to $\prod_{e\in C} \tau(e) \in \{\pm 1\}$ is a group homomorphism from the cycle space of Q_n (over GF(2), under symmetric difference) to $\{\pm 1\}$, since edges shared by two cycles cancel in their symmetric difference and likewise cancel in the product. The squares of Q_n generate its cycle space (a standard fact for hypercubes: every cycle is a symmetric difference of squares; we independently verified this by direct GF(2) rank computation for every n used in this paper, $n = 3, \ldots, 8$ – cycle space dimension $E - V + 1$ equal to the rank of the square vectors in every case, including $n = 7$'s dimension 321 and $n = 8$'s dimension 769), so this homomorphism is determined by its values on the squares; since it is $+1$ on every square, it is $+1$ on every cycle. A signature with product $+1$ on every cycle of a connected graph is a *coboundary*: $\tau(x,y) = t(x)t(y)$ for some $t : V \to \{\pm 1\}$ (the classical balance theorem for signed graphs, Harary [7]). Solving for σ_2 gives the claim. □

Lemma 8 says any two odd-square signatures are related by a *vertex switching* t. For a *fixed* coupling J, letting the spin configuration $s : V(Q_n) \to \{\pm 1\}$ range over all 2^{2^n} choices (one sign per vertex of Q_n, which itself has 2^n vertices) produces exactly the signatures $\sigma_s(x,y) = J(x,y)s(x)s(y)$ in the switching class of J (writing $s = t \cdot s_0$ for a fixed reference s_0 realises every switching t); this is the operational fact we use, not a claim that different s give the same energy under fixed J (they generally do not – that variation is exactly what is being optimised). The energy-preserving gauge transformation in the physical sense instead acts on *both* J and s together, $J(x,y) \mapsto J(x,y)t(x)t(y)$ and $s(x) \mapsto t(x)s(x)$, under which $J(x,y)s(x)s(y)$ is unchanged edge-by-edge [11]. Consequently, $g(n)$ — defined as a maximum over *all* odd-square edge sets — equals the maximum of $|E|$ over the switching class of any one fixed fully-frustrated signature, in particular the reference coupling J used in Section 5.3's witness; optimising that one fixed J (or, for $D = 8$, exhibiting a ground state attaining Derrida et al.'s improved energy lower bound for it, as Marinari–Parisi–Ritort report [11]) therefore does optimise over the class defining $g(n)$, not merely over a possibly-proper subclass of it.

Under the further correspondence between signed hypercubes and Ising spin configurations $s : \{0,1\}^n \to \{\pm 1\}$ via $\sigma(x,y) = J(x,y)\, s(x)s(y)$ for a fixed reference coupling

J with every square frustrated, maximising $|E|$ (i.e. maximising the number of *positive* edges, equivalently minimising the number of negative edges) is exactly the problem of minimising the ground-state energy of the fully frustrated Ising hypercube studied by Derrida, Pomeau, Toulouse, and Vannimenus [5] and, for $D = 8$, by Marinari, Parisi, and Ritort [11]: the ground-state energy is (up to an affine rescaling) the number of negative edges, $|E(Q_n)| - |E|$. In the language of signed graphs, $|E(Q_n)| - g(n)$ is the *frustration index* of the fully frustrated signature on Q_n. Laplante, Naserasr, Sunny, and Wang [10] study this frustration index directly (in the context of Huang's proof of the sensitivity conjecture) and explicitly establish a connection between it and Erdős's question on the largest C_4-free subgraph of the hypercube, giving lower and upper bounds on the frustration index of the fully frustrated signature that match when n is a power of 4; our $g(6), g(7), g(8)$ results above sharpen this connection to exact values at $n = 6, 7, 8$ specifically. This sign convention matches the one used by the reconstruction script `generate_q8_682.py` (Section 5.3), where E is exactly the set of edges with $J(x, \mathrm{dim})\, s(x)s(y) = +1$; for a vertex v of degree $\deg_E(v)$ in E, we call $h(v) = 2\deg_E(v) - n$ its *local field* under this convention (matching the local-field histogram reported in Proposition 13 below). We do not reproduce Derrida et al.'s derivation of the local-field lower bound here, and do not need to: Section 5.2 derives the bounds it uses from Lemmas 9 and 11 directly, and cites [5, 11] only for historical priority. Their results are stated there for reference — established for $D \leq 7$ by explicit construction, and used by Marinari–Parisi–Ritort at $D = 8$ — and the corresponding witnesses are constructed and verified independently in Section 5.3.

This connection was brought to our attention, after the original circulation of this manuscript, by S. Lai (see Acknowledgements), who identified the relevance of [5, 11] and located the improved Q_8 result.

5.2 Exact values for $n = 6, 7, 8$

Derrida et al. [5] give a local-field lower bound on the number of negative (frustrated, in their terminology) edges, tight enough to determine the fully frustrated hypercube's ground-state energy exactly for $D \leq 7$ via an explicit recursive construction, and conjectured it remains tight for $D \geq 8$. For $D = 6$, they give this explicit construction directly: "this configuration for H_6 contains 60 negative bonds (out of a total of 192)" [5, p. 622], i.e. $192 - 60 = 132$ positive (unfrustrated) edges, with the construction guaranteed fully frustrated (every square meeting it in an odd number of negative edges) by their general recursive method. Under the correspondence of Section 5 this is an explicit odd-square witness with 132 edges, so $g(6) \geq 132$. We cite this for historical priority only: the lower bound $g(6) \geq 132$ is established below by an explicit odd-square edge list of our own (Section 5.3, `q6_odd_square_132.json`), and the matching upper bound $g(6) \leq 132$ follows from Lemma 9 and Lemma 11 without appeal to [9]. We did not reconstruct DPTV's specific $D = 6$ bond configuration; our witness is an independent one, which however turns out to realise the same local-field distribution they report (see Section 5.3).

Lemma 9 (Self-contained bound for $n = 7, 8$)**.** *For any odd-square edge set E in Q_n, writing $U = \{v \in \{0,1\}^n : v$ has an even number of 1-bits$\}$ for the even-parity part of the bipartition of Q_n ($|U| = 2^{n-1}$; the argument below is symmetric in U and its complement, so this choice is only for concreteness and all numerical values reported for U throughout this paper, e.g. Proposition 13's h-distribution, use this same convention) and $h(v) = 2\deg_E(v) - n$ as before, every $h(v)$ with $v \in U$ is an integer of the same parity* *as n, and*

$$\sum_{v\in U} h(v)^2 = n\cdot 2^{n-1}.$$

Proof. For $v\in U$ and direction $i\in\{0,\dots,n-1\}$ write $s_i(v)=\sigma(v,v\oplus e_i)\in\{\pm 1\}$, so $h(v)=\sum_i s_i(v)$ and $h(v)^2 = n+\sum_{i\neq j} s_i(v)s_j(v)$. Fix $i\neq j$ and consider, for $v\in U$, the square on $\{v, v\oplus e_i, v\oplus e_j, z\}$ where $z=v\oplus e_i\oplus e_j\in U$. Its four edges have signs $s_i(v)$, $s_j(v)$, and (computed from z's own perspective, since a sign is a single scalar attached to the edge) $s_j(z)$, $s_i(z)$. The odd-square condition means their product is -1: $s_i(v)s_j(v)\cdot s_j(z)s_i(z)=-1$. Writing $X=s_i(v)s_j(v)$ and $Y=s_i(z)s_j(z)$, both lie in $\{\pm1\}$ and $XY=-1$ forces $X=-Y$, i.e. $s_i(v)s_j(v)+s_i(z)s_j(z)=0$. The map $v\mapsto z=v\oplus e_i\oplus e_j$ is a fixed-point-free involution on U (as $e_i\oplus e_j\neq 0$), so it partitions U into pairs $\{v,z\}$ on each of which the two corresponding terms cancel; summing over all such pairs gives $\sum_{v\in U} s_i(v)s_j(v)=0$ for every fixed $i\neq j$. Summing over all ordered pairs $i\neq j$ and adding the $n\cdot|U|$ diagonal contribution gives $\sum_{v\in U} h(v)^2 = n\cdot 2^{n-1}$. □

This proof uses only that *every* square has product -1 (the odd-square condition), so it is sufficient for the identity to hold; it is not necessary. Testing against all 19 866 published 304-edge Q_7 solutions – odd-square and non-odd-square alike – shows every one of them satisfies $\sum_{v\in U} h(v)^2 = 448$, including the 19 477 that are not odd-square. This is explained, for these particular solutions, by a simpler fact that does not need odd-squareness either: every one of the 19 866 has all degrees in $\{4,5\}$ (Proposition 16), and since Q_7 is bipartite, $\sum_{v\in U}\deg(v)=|E|=304$ exactly (every edge has exactly one endpoint in U); writing a,b for the number of degree-4, degree-5 vertices in U ($a+b=64$), $4a+5b=304$ forces $b=48, a=16$ regardless of which specific 304-edge solution is chosen, so $h(v)=2\deg(v)-7\in\{1,3\}$ takes value 1 on exactly 16 vertices of U and 3 on the other 48, giving $\sum_{v\in U} h(v)^2 = 16\cdot 1^2+48\cdot 3^2 = 16+432=448$ automatically. We verified the U-side split is indeed exactly $\{4^{16},5^{48}\}$ for all 19 866, with no exceptions. The identity's failure for the non-odd-square Q_8 Solution B of Section 4 ($1016\neq 1024$) is therefore not explained by odd-squareness as such: Solution B has an asymmetric degree distribution between U and its complement ($\{4^3,5^{82},6^{43}\}$ vs. $\{4^4,5^{80},6^{44}\}$), whereas Solution A, all 19 866 Q_7 solutions, and the 682-edge witness all have symmetric U/complement degree distributions. This symmetry is not itself implied by odd-squareness, however: a small odd-square counterexample in Q_3 (vertices $0,\dots,7$, $E=\{(0,2),(2,6),(3,7),(4,5),(4,6),(6,7)\}$, whose six squares meet E, in the direction-pair-then-base enumeration order of Lemma 3, in $1,3,1,3,3,1$ edges) has U-degree multiset $\{1,1,1,3\}$ against $\{0,2,2,2\}$ on the complement, while still satisfying $\sum_{v\in U} h(v)^2=\sum_{v\in U^c} h(v)^2 = 12 = 3\cdot 2^2$ on each side separately. We do not characterise the identity's full necessary-and-sufficient condition here.

Lemma 10 (Nonnegativity at the maximum)**.** *If E is odd-square in Q_n with $|E|$ maximum (i.e. $|E|=g(n)$), then $h(v)\geq 0$ for every $v\in\{0,1\}^n$.*

Proof. Suppose $h(v)<0$ for some v. Flipping the spin at v (replacing $s(v)$ by $-s(v)$, all other spins unchanged) flips $\sigma(v,w)$ for every edge (v,w) incident to v and leaves every other edge's sign unchanged. Every square containing v has exactly two edges incident to v; flipping both leaves their product, and hence the whole square's sign product, unchanged, so odd-squareness is preserved everywhere. At v itself, the positive-edge count changes from $p=(n+h(v))/2$ to $q=(n-h(v))/2$, a change of $q-p=-h(v)>0$; no edge other than the n edges incident to v changes sign, so $|E|$ increases by exactly $-h(v)>0$. This contradicts maximality of $|E|$, so no such v exists. □

This is a corollary of, not an alternative to, Lemma 8: the argument uses only that flipping one vertex's spin preserves odd-squareness (immediate from the switching correspondence) and counts the resulting change in $|E|$ directly. All 389 released odd-square Q_7 solutions have 304 edges, the maximum shown below (Section 6.4), so the Lemma's hypothesis applies to every one of them; we independently verified $h(v) \geq 0$ for all $v \in U$ holds on all 389, with no counterexample, consistent with (and not merely assumed by) the Lemma.

Given Lemma 9 alone, $|E| = (n \cdot 2^{n-1} + \sum_{v \in U} h(v))/2$ is bounded above by the integer optimisation problem of maximising $\sum_{v \in U} h(v)$ subject to the *fixed* sum of squares $\sum_{v \in U} h(v)^2 = n \cdot 2^{n-1}$ over the 2^{n-1} integers $h(v)$ of fixed parity (we use only that every realisable h-vector satisfies these constraints, not the converse that every integer vector satisfying them is realisable as an odd-square signature; the bound obtained is therefore an upper bound on $|E|$, tight whenever a matching witness is exhibited). No sign hypothesis enters: the constraints admit negative h, and Lemma 10 is *not* used in the derivation. Its role is the separate one of describing configurations that actually attain the maximum (matching what we verified computationally on all 389 odd-square Q_7 witnesses above).

The following elementary observation is what makes this integer problem solvable in closed form, and is the only ingredient beyond the classical two-point convexity inequality.

Lemma 11 (Quantised two-point inequality)**.** *Let a be an integer of the same parity as n and put $b = a + 2$. Then for every integer h of the same parity as n,*

$$(h - a)(h - b) \;\geq\; 0 \qquad \textit{and} \qquad (h - a)(h - b) \equiv 0 \pmod{8}.$$

In particular $(h - a)(h - b) \in \{0, 8, 24, 48, \ldots\}$, so it is either 0 (exactly when $h \in \{a, b\}$) or at least 8.

Proof. $h - a$ is even, say $h - a = 2u$ with $u \in \mathbb{Z}$; then $h - b = 2u - 2$, so $(h - a)(h - b) = 4u(u - 1)$. Consecutive integers $u, u - 1$ have product ≥ 0 and even, so $4u(u - 1)$ is a nonnegative multiple of 8. It vanishes exactly when $u \in \{0, 1\}$, i.e. $h \in \{a, b\}$. ☐

Summing Lemma 11 over $v \in U$ and writing $S = \sum_{v \in U} h(v)$, $m = |U| = 2^{n-1}$, $Q = \sum_{v \in U} h(v)^2 = n \cdot 2^{n-1}$ gives the single inequality used for all three values of n:

$$\Delta \;:=\; \sum_{v \in U} (h(v) - a)(h(v) - b) \;=\; Q - (a + b)S + ab\,m \;\geq\; 0, \qquad \Delta \equiv 0 \pmod{8}. \tag{2}$$

The nonnegativity is the classical bound $S \leq (Q + ab\,m)/(a + b)$; the congruence is what removes the residual gap between that real bound and the true integer optimum. We apply (2) with the pair (a, b) of parity-correct integers straddling the root-mean-square field $\sqrt{Q/m} = \sqrt{n}$, namely $(1, 3)$ for $n = 7$ ($\sqrt{7} \approx 2.65$) and $(2, 4)$ for $n = 6, 8$ ($\sqrt{6} \approx 2.45$, $\sqrt{8} \approx 2.83$); this is the choice minimising the real bound $(Q + ab\,m)/(a + b)$.

$n = 6$. Here $m = 32$, $Q = 192$, $(a, b) = (2, 4)$, so $\Delta = 448 - 6S$. Nonnegativity alone gives only $S \leq 74.\overline{6}$, hence $S \leq 74$ by parity; but $448 \equiv 0 \pmod{8}$ forces $6S \equiv 0 \pmod{8}$, i.e. $S \equiv 0 \pmod{4}$. The largest such S below $74.\overline{6}$ is

$$S \leq 72, \qquad \text{whence} \qquad g(6) \leq \frac{192 + 72}{2} = 132.$$

At $S = 72$ we have $\Delta = 16$, and since each vertex contributes 0 or at least 8, exactly two vertices of U contribute 8 each, i.e. exactly two have $h \in \{0, 6\}$ and the remaining 30 have

$h \in \{2, 4\}$. The witness of Section 5.3 attains $S = 72$ with h-distribution $\{0{:}2,\ 2{:}24,\ 4{:}6\}$ on U, so the bound is tight and $g(6) = 132$ is established without external input — in particular without [9], which we retain only for the unrestricted value $\mathrm{ex}(Q_6, C_4) = 132$ (Section 8).

$n = 7$. Here $m = 64$, $Q = 448$, $(a, b) = (1, 3)$, so $\Delta = 640 - 4S \geq 0$ gives $S \leq 160$, attained with $\Delta = 0$. By Lemma 11 equality $\Delta = 0$ forces $h(v) \in \{1, 3\}$ at *every* $v \in U$; combined with $\sum h = 160$ and $|U| = 64$ this pins the distribution to $\{1{:}16,\ 3{:}48\}$ uniquely. Hence

$$g(7) \leq \frac{448 + 160}{2} = 304.$$

Since $|E| = (n \cdot 2^{n-1} + \sum_{v \in U} h(v))/2$ for *any* 304-edge set (odd-square or not), attaining $S = 160$ is automatic and does not by itself exhibit an odd-square witness; what is needed is an explicit odd-square set achieving it. The odd-square audit (Section 6.4) supplies 389 such witnesses among the released 304-edge Q_7 solutions; the first in file order is global index 34 (`q7_edges_304.jsonl.part1`, line 35; SHA-256 below is of the canonicalised `edges` array – matching `q7_odd_square_389.csv`'s `solution_sha256` column, not the JSONL line or any `hash` field: `2f2649ae9c4d1c901ba5a88f9825689de3cc19de1b742d86f5d66dbfd3aebec7`), with square-intersection histogram $\{1{:}96,\ 3{:}576\}$, confirming it is genuinely odd-square and attains $S = 160$. (The file's very first record, by contrast, has histogram $\{1{:}36,\ 2{:}120,\ 3{:}516\}$ and is *not* odd-square, despite also satisfying $S = 160$ – exactly because that identity holds regardless of odd-squareness.) So $g(7) = 304$.

$n = 8$. Here $m = 128$, $Q = 1024$, $(a, b) = (2, 4)$, so $\Delta = 2048 - 6S$. Nonnegativity gives $S \leq 341.\overline{3}$; the congruence $2048 \equiv 0 \pmod 8$ again forces $S \equiv 0 \pmod 4$, so

$$S \leq 340, \qquad \text{whence} \qquad g(8) \leq \frac{1024 + 340}{2} = 682.$$

At $S = 340$ we have $\Delta = 8$: *exactly one* vertex of U contributes, and it has $h \in \{0, 6\}$, the other 127 having $h \in \{2, 4\}$. Pointwise equality $h(v) \in \{2, 4\}$ everywhere is therefore *impossible* at the optimum for $n = 8$ — in contrast with $n = 7$, where $\Delta = 0$ does force it. The released 682-edge witness attains $S = 340$ with h-distribution $\{0{:}1,\ 2{:}84,\ 4{:}43\}$ on U (independently recomputed from the released edge list), consistent with this description, so the bound is tight and $g(8) = 682$.

We stress that Lemma 11 solves these three instances exactly, not merely up to rounding: an independent exhaustive solution of the integer programme $\max\{\sum h : \sum h^2 = n \cdot 2^{n-1},\ h \equiv n\ (2),\ |h| \leq n\}$ by dynamic programming over the sum-of-squares budget returns optima 72, 160, 340 for $n = 6, 7, 8$ respectively, matching the bounds above, and returns exactly 3, 1 and 2 optimal h-distributions respectively — for $n = 6$, $\{2{:}30, 6{:}2\}$, $\{0{:}1, 2{:}27, 4{:}3, 6{:}1\}$ and $\{0{:}2, 2{:}24, 4{:}6\}$; for $n = 7$, $\{1{:}16, 3{:}48\}$; for $n = 8$, $\{2{:}87, 4{:}40, 6{:}1\}$ and $\{0{:}1, 2{:}84, 4{:}43\}$. In each case the witness released with this paper realises one of them. This computation is reproducible with the bundled `solve_field_ip.py` (standard library only, seconds to run); it is a check on the closed-form argument above, not a substitute for it.

All three upper bounds $g(6) \leq 132$, $g(7) \leq 304$ and $g(8) \leq 682$ are thus established directly from Lemma 9 and Lemma 11, independently of DPTV's Table I and of [9]; we retain the citations to Derrida et al. [5] and Marinari–Parisi–Ritort [11] for historical priority and for the connection that prompted this section, not because the upper-bound proof depends on them.

Marinari, Parisi, and Ritort [11] report having exhibited, by simulated annealing, a $D = 8$ ground state attaining Derrida et al.'s improved energy lower bound. Their attainment claim, under the correspondence developed below, corresponds to the same bound value $\sum_{v\in U} h(v) = 340$ established directly above – MPR95 do not themselves report a quantity called S or 340; the correspondence is ours. Their Hamiltonian is $H \equiv -D^{-1/2}\sum_{(x,y)} J_{x,y}s_x s_y$ [11, Eq. 3]; writing $S = \sum_{(x,y)} J_{x,y}s_x s_y$ for the sum over *all* edges (not just U), a bipartite argument identical to the one used for Lemma 9 shows $S = \sum_{v\in U} h(v)$ exactly (every edge has exactly one endpoint in U, so summing the local field over U counts each edge's sign once), so $H = -S/\sqrt{D}$ and, per site, $H/N = -S/(N\sqrt{D})$ with $N = 2^D$; at $D = 8, S = 340$ this is $H \approx -120.2$, or $H/N \approx -0.470$ per site. As a cross-check against MPR95's own text: they report (in their D=9 discussion) that "on a scale where the minimal energy jump is 1" their D=9 naive bound $H/N = -1/2$ corresponds to an integer value of -384 (i.e. a scale factor of $768 = 384/(1/2)$), and state that on this same scale "the improved lower bound at $D = 8$ gives -361" – note this 768 factor is MPR95's own D=9 comparison scale, not an intrinsic D=8 energy-quantisation unit; they use it purely to compare bound values across dimensions on one common integer axis. Applying the same factor (as MPR95 themselves do for the $D = 8$ figure) to our value gives $768 \times (-0.4696\ldots) = -360.62\ldots \approx -361$, matching their reported integer to within rounding. Their paper reports the $D = 8$ attainment in energy units only, without an explicit spin configuration or its translation into edge-count terms; we supply that translation and an explicit, machine-verified 682-edge certificate in Section 5.3 below. Under the correspondence of Section 5, this historical attainment report at $D = 8$ corresponds to $g(8) = 682$, matching what was already established above from Lemma 9 together with the integer/parity constraints and elementary inequalities alone (as detailed above, Lemma 10 is not needed for this bound); the explicit witness realising it is due to the present reconstruction, prompted by S. Lai's identification of the connection to [5, 11]. Together with the matching witnesses at $n = 6, 7$ this gives Theorem 2: $g(6) = 132$, $g(7) = 304$, and $g(8) = 682$, each half of each equality now carried by material released with this paper.

5.3 Verification

We constructed explicit witnesses for all three values and machine-verified them independently of [5, 11, 9].

Q_6, $g(6) = 132$. We obtained a 132-edge odd-square witness for Q_6 by the same route used for Q_8 below: the canonical fully frustrated coupling

$$J(x, \mathrm{dim}) = (-1)^{\mathrm{popcount}(x\,\&\,(2^{\mathrm{dim}}-1))},$$

verified fully frustrated on Q_6 (all 240 squares have coupling product -1, no exceptions), together with a 64-bit spin configuration. Because switching preserves frustration, the set of edges on which $J(x, y)s_x s_y = +1$ is odd-square for *every* spin assignment, so the construction reduces to maximising that edge count, i.e. to a ground-state search. We ran a fixed-seed simulated annealing over the 64 spins (`search_q6.py`, seed 20260823), which reached 132 edges on its first trial; the run is exactly reproducible from the released script. The resulting witness is regenerated and self-checked by `generate_q6_132.py` and released as `q6_odd_square_132.json`.

Proposition 12. *The Q_6 witness edge set satisfies:*

(i) 132 *edges, all valid Q_6 edges, no repetitions.*

(ii) Square-intersection histogram $\{1\!:\!30,\ 3\!:\!210\}$ *over all* 240 *squares: the odd-square condition holds everywhere, and in particular the set is* C_4*-free.*

(iii) Degree sequence $\{3^4,\ 4^{48},\ 5^{12}\}$*; degree sum* $264 = 2 \cdot 132$*.*

(iv) Local-field histogram on U *equal to* $\{0\!:\!2,\ 2\!:\!24,\ 4\!:\!6\}$*, giving* $\sum_{v\in U} h(v) = 72$ *and* $\sum_{v\in U} h(v)^2 = 192 = 6 \cdot 2^5$*, as required by Lemma 9; the complementary side of the bipartition has the same histogram.*

(v) Local maximality: no edge of Q_6 *can be added to it while preserving* C_4*-freeness.*

The distribution in (iv) is one of the exactly three optimal h-distributions enumerated in Section 5.2, so the witness attains the upper bound $S \le 72$ there and settles $g(6) = 132$. Two further remarks. First, it coincides in multiplicities with the local-field distribution Derrida et al. report for their $D = 6$ configuration, "$n(4) = 6$; $n(2) = 24$; $n(0) = 2$" [5, p. 622]; we did not reconstruct their configuration and do not claim the two edge sets agree, only that our independently found witness realises the same distribution. Second, this witness is *not* the released 132-edge Q_6 construction `q6_edges_132.jsonl` of Section 8: the two edge sets differ in 62 edges, and the latter is C_4-free but not odd-square (square-intersection histogram $\{1:8,\ 2:44,\ 3:188\}$, and $\sum_{v\in U} h(v)^2 = 176 \neq 192$). Both are 132-edge C_4-free subgraphs of Q_6; only the new one is a witness for $g(6)$ specifically.

Q_7, $g(7) = 304$. The 304-edge odd-square value coincides with our originally announced Q_7 lower bound (Theorem 1(i)); of the 19 866 solutions obtained via the SA-seed-plus-collector pipeline (Section 3), 389 are themselves odd-square witnesses of $g(7) = 304$ (Section 6), independently confirming reachability of this value without reference to [5].

Q_8, $g(8) = 682$. We obtained a 682-edge odd-square witness for Q_8 from an explicit 256-bit spin configuration together with the canonical fully frustrated coupling $J(x, \mathrm{dim}) = (-1)^{\mathrm{popcount}(x \,\&\, (2^{\mathrm{dim}}-1))}$ (one coupling convention realising a fully frustrated signature; see Section 5 supplementary script `generate_q8_682.py`). The script is dependency-free (Python standard library only), recomputes the edge set from the spin configuration, and asserts every structural quantity below against the recomputed data before writing the certificate; it does not take any of these quantities as input. *Provenance of the spin configuration.* The 256-bit configuration was derived by the author on 17–18 August 2026, using the canonical fully frustrated coupling of Marinari, Parisi, and Ritort [11], as recorded in dated correspondence with S. Lai, who independently reconstructed the identical 682-edge set from the shared spin vector and cross-checked all 1,792 squares, the degree distribution, and the local-field distribution before this material was incorporated into the manuscript. This provenance record is a first-person, dated account (the correspondence itself), not an independently reproducible computational log: no search seed, intermediate search records, or derivation script for the spin configuration itself is included in the released materials, only the scripts that regenerate and verify the edge set *from* the given spin configuration (Section 5). We do not claim this configuration is identical to any specific configuration MPR may have used internally, since they did not publish one; it is an independently derived witness attaining the same bound.

Proposition 13. *The witness edge set satisfies:*

(i) 682 *edges, spanning all* 256 *vertices of* Q_8*.*

(ii) Square-intersection histogram $\{1\!:\!301,\ 3\!:\!1491\}$ *over all* 1 792 *squares (odd-square condition holds everywhere; zero squares meet the edge set in* 0*,* 2*, or* 4 *edges).*

(iii) Degree sequence $\{4^2, 5^{168}, 6^{86}\}$*; degree sum* $1364 = 2 \cdot 682$*.*

(iv) Local-field histogram $\{0\!:\!2,\ 2\!:\!168,\ 4\!:\!86\}$ *over all* 256 *vertices, where the local field at a vertex is (positive degree) − (negative degree) under the signed-graph correspondence above; restricted to* U *specifically (the* 128 *vertices summed over in Lemma 9 and the upper-bound derivation of Section 5.2), the histogram is* $\{0\!:\!1,\ 2\!:\!84,\ 4\!:\!43\}$*, giving* $\sum_{v \in U} h(v) = 2 \cdot 84 + 4 \cdot 43 = 340$ *directly.*

(v) Local maximality: every one of the 342 *non-edges of* Q_8 *creates at least* 3 *new* C_4*s upon addition (distribution* $\{3\!:\!41,\ 4\!:\!159,\ 5\!:\!120,\ 6\!:\!22\}$*), a strictly stronger local-maximality margin than the originally announced 680-edge Solution B of Section 4, for which 2 of 344 non-edges created only 1 or 2 new* C_4*s.*

All quantities in Proposition 13 were recomputed independently of the reconstruction script by direct enumeration of all 1 792 squares against the released edge list, using the strict definition of C_4-freeness from Lemma 3 (no square may have all four edges present); this independent check found zero violations and is fully reproducible from the released files alone. The reconstruction, its SHA-256 checksum, and an independent audit are additionally reported to have been cross-checked against S. Lai's separately produced implementation, with full agreement; this cross-check is recorded as a dated correspondence account (as in Section 5.3), not as an independently verifiable artefact, since the correspondence, Lai's implementation, and its output are not included in the released materials.

Table 3 summarises the odd-square reconstructions alongside the corresponding simulated-annealing witnesses of Sections 3 and 4.

Table 3: Odd-square reconstructions compared with the simulated-annealing witnesses of Sections 3–4.

Parameter	Q_7 odd-square	Q_8 odd-square
Edges (value of $g(n)$)	304	682
Source	one of 389 verified SA sols.	witness for [11] bound (reconstructed here)
Non-odd-square comparison witness	n/a (19 477 exist, 6.4)	680 (Solution B, 4)
Improvement over that witness	—	+2

6 Structural Classification of Q_7 Solutions

6.1 Dimension profiles and the twenty types

Definition 14 (Dimension profile)**.** For $G \subseteq Q_7$, let $e_i(G) = |\{uv \in E(G) : u \oplus v = 2^i\}|$. The *dimension profile* is the sorted tuple $\pi(G) = (e_{\sigma(0)} \geq \cdots \geq e_{\sigma(6)})$.

The dimension profile is invariant under coordinate permutations (a subgroup of $\mathrm{Aut}(Q_7)$ of order $7! = 5040$), giving a coarse but computable classification.

Proposition 15. *Among the* 19 866 *solutions, the dimension profile takes exactly* **20** *distinct values.*

Table 4: The 20 dimension-profile types of 304-edge C_4-free subgraphs of Q_7, sorted by decreasing frequency. H: Shannon entropy of normalised profile (bits).

Rank	Profile π	Solutions	H	λ_1 (mean)
1	$[44, 44, 44, 44, 43, 43, 42]$	3155	2.8072	4.787
2	$[45, 45, 45, 43, 43, 42, 41]$	2913	2.8065	4.788
3	$[45, 45, 45, 43, 43, 43, 40]$	2200	2.8063	4.787
4	$[44, 44, 44, 44, 44, 43, 41]$	2116	2.8069	4.787
5	$[46, 46, 46, 42, 42, 42, 40]$	1756	2.8053	4.787
6	$[46, 46, 46, 42, 42, 41, 41]$	1181	2.8054	4.789
7	$[44, 44, 44, 44, 44, 44, 40]$	1085	2.8066	4.787
8	$[45, 45, 45, 43, 42, 42, 42]$	1064	2.8066	4.788
9	$[45, 45, 44, 43, 43, 43, 41]$	974	2.8067	4.787
10	$[44, 44, 44, 44, 44, 42, 42]$	931	2.8070	4.787
11	$[47, 47, 47, 41, 41, 41, 40]$	902	2.8037	4.788
12	$[45, 45, 43, 43, 43, 43, 42]$	439	2.8069	4.788
13	$[45, 44, 44, 43, 43, 43, 42]$	307	2.8070	4.788
14	$[46, 45, 45, 42, 42, 42, 42]$	236	2.8063	4.789
15	$[44, 44, 44, 43, 43, 43, 43]$	163	2.8073	4.787
16	$[47, 47, 44, 43, 41, 41, 41]$	153	2.8050	4.788
17	$[46, 46, 44, 42, 42, 42, 42]$	107	2.8062	4.789
18	$[48, 48, 48, 40, 40, 40, 40]$	101	2.8014	4.788
19	$[46, 46, 43, 43, 42, 42, 42]$	44	2.8063	4.788
20	$[46, 46, 45, 42, 42, 42, 41]$	39	2.8058	4.788
	Total	**19866**		

Table 4 lists all 20 types with frequencies and structural invariants.

Observations: (1) the profile-wise *mean* values of λ_1 shown in the table lie in $[4.787, 4.789]$ across all 20 types (this is distinct from, and narrower than, the exhaustive range $[4.78543, 4.79129]$ over all 19 866 individual solutions reported in Proposition 16 below). (2) H ranges from 2.801 to 2.807, all near $\log_2 7 \approx 2.807$. (3) All 101 Type-18 solutions have a genuine $\mathrm{Aut}(Q_7)$-automorphism (coordinate permutation combined with a bitwise XOR, i.e. the full automorphism group $\mathrm{Aut}(Q_7) \cong (\mathbb{Z}_2)^7 \rtimes S_7$ of Q_7 itself – a proper subgroup of the full affine group $\mathrm{AGL}(7, 2)$ over $\mathbb{F}_2^7$, since only coordinate *permutations* are allowed, not general linear maps – not merely a coordinate permutation) that nontrivially permutes at least two of the three tied 48-edge directions; of these, 46/101 have a genuine order-3 automorphism cyclically permuting all three directions. (An earlier check that considered only pure coordinate permutations, without combining them with the XOR/translation part of $\mathrm{Aut}(Q_7)$, incorrectly found no such automorphisms in a 20-solution sample; that check was in error and has been redone exhaustively over all 101 solutions.)

6.2 Shared structural properties

Proposition 16. *Every solution in the* 19 866 *satisfies:*

(i) Degree sequence $\{4^{32}, 5^{96}\}$.

(ii) λ_1 *ranges over* $[4.78543, 4.79129]$ *across all* 19 866 *solutions (exhaustively computed; not a single shared three-decimal value), with* $\lambda_1 + \lambda_n = 0$ *throughout, automatic for*

any subgraph of the bipartite Q_7 (sanity check, not an independent finding). The type-wise means of Table 4 round to 4.787*–*4.789*.*

(iii) $\mathrm{Tr}(A^4) = 5216$ *(C_4-free trace equality).*

(iv) *Local maximality: no edge of Q_7 can be added without creating a C_4.*

(v) *Every edge of Q_7 appears in at least one solution (verified by taking the union of the edge sets over all* 19 866 *solutions).*

6.3 Solution landscape

Pairwise Hamming distances $d_H(G, G') = |E(G) \triangle E(G')|$, computed exhaustively over all $\binom{19\,866}{2} = 197\,319\,045$ pairs, range from **6** (attained by a pair of profile-type-1 and profile-type-2 solutions, differing by just 3 edges each way) to **274**, with exact mean 195.396108 . . . and exact median 196 (all four values computed directly from the full pairwise distance distribution, not sampled).

For historical completeness only, not as a reviewable computational result: a failed search targeting 305 edges was also reported.[2]

Conjecture 17. $\mathrm{ex}(Q_7, C_4) = 304$.

6.4 Odd-square audit

We audited all 19 866 solutions of Section 3 against the odd-square condition of Section 5 (every square meets the solution in exactly one or three edges). Exactly 389 solutions are odd-square; the remaining 19 477 are not. Table 5 gives the dimension-profile breakdown of the 389 odd-square solutions; each belongs to one of only 4 of the 20 profile types of Table 4 (ranks 5, 7, 10, and 18 there).

Table 5: Dimension-profile breakdown of the 389 odd-square solutions among the 19 866 published 304-edge Q_7 solutions.

Dimension profile (sorted)	Count
$\{44, 44, 44, 44, 44, 44, 40\}$	127
$\{44, 44, 44, 44, 44, 42, 42\}$	83
$\{48, 48, 48, 40, 40, 40, 40\}$	101
$\{46, 46, 46, 42, 42, 42, 40\}$	78
Total odd-square	389
Total non-odd-square	19 477

Since the odd-square subclass accounts for only $389/19\,866 \approx 2.0\%$ of the published solutions, the shared structural core of Proposition 16 and the 20-type classification of

[2]Over 1 076 reported trials targeting 305 edges, the minimum C_4 violation count was 2 (never 0). This figure is identical to the 681-edge Q_8 statistic reported in Section 4; we do not have a log or seed list for either figure, so we cannot confirm whether this is a genuine coincidence across two separate search efforts or a duplication error somewhere in this manuscript's history. We did not attempt to resolve this by guessing; both figures are unarchived, author-recorded claims of uncertain independence from each other, mentioned only as development history and bearing on no claim in this paper.

Section 6.1 are not fully explained or subsumed by the odd-square construction: the great majority of 304-edge C_4-free subgraphs of Q_7 found here lie outside the fully frustrated switching class of [5]. The one exception is Type 18 ($[48, 48, 48, 40, 40, 40, 40]$, 101 solutions), all of which are odd-square; the other three types containing odd-square solutions (ranks 5, 7, and 10) are mixed, containing both odd-square and non-odd-square members.

7 Computational Method

The successful outcomes described in this section (the released edge sets themselves) are machine-verifiable certificates: any reader can confirm C_4-freeness and the reported edge counts directly from the supplied data with `verify.py`, independently of how the search was run. The Hamming-distance statistics of Section 6.3 are likewise on a different footing from the search-process claims below: they are computed exhaustively over all released solutions, not sampled, so they are recomputable directly from the released edge lists alone. We release `analyze_q7_structure.py` alongside this revision, which recomputes this section's degree-sequence, dimension-profile, spectral-radius-range, exhaustive Hamming-distance, and Type-18 nontrivial-automorphism-existence claims directly from `q7_edges_304.jsonl.part1--3` in about three minutes (peak memory approximately 100MB); running it reproduces all of those specific numbers exactly. This script confirms only that *some* nontrivial automorphism fixes each Type-18 solution; it does not check the stronger claim made in Section 6 that such an automorphism nontrivially permutes at least two of the three tied 48-edge directions, nor does it determine automorphism *order*: both the direction-permuting property and the 46/101 order-3 figure elsewhere in this paper were established by a separate, more detailed check (verifying the automorphism's action on directions, and $\sigma^3 = \mathrm{id} \neq \sigma, \sigma^2$, directly for each automorphism found), not included in this script.
Update: the Q_7 production code has been recovered and verified, with one important caveat. Earlier versions of this manuscript reported the Q_7 provenance question as unresolved, based on the released `c4free_sa.py` (Section 7.2 below), whose `phase1_sa` we found cannot progress from a fresh random start under its documented parameters. We have since located scripts that produce the 19 866 released Q_7 solutions: `sa_search.py` (a conventional multi-move-type penalty SA – add, remove, 1-swap, and kick moves, with a proper temperature-scaled Boltzmann acceptance rule rather than a hard violation cap) and `sa_collect304.py` (a remove-and-repair collector: starting from an existing valid 304-edge solution – either the seed or a previously found solution, optionally first transformed by a random element of $\mathrm{Aut}(Q_7)$ – it removes a small number k of edges, drawn from $\{1, 2, 3, 4, 5, 8, 12, 20\}$ with fixed weights, then greedily re-adds edges one at a time, accepting only additions that create zero new violating squares, keeping the result if it reaches 304 edges with zero violations and has not been seen before). The caveat: the released `sa_search.py` unconditionally loads its starting state from `selected_edges_best.json` (no random/greedy fallback), and the specific file we recovered and release alongside this revision already contains the final 304-edge solution; running the released script therefore searches for *improvements beyond* 304 edges (consistent with the later, separately-recovered 305-edge search history discussed below), not a from-scratch discovery of the first 304-edge solution. We have since traced the entire earlier chain, with one significant correction to an earlier draft of this section. The earliest recovered attempt on this problem, dated over a week before `selected_edges_301.json` and every other file discussed above, is a CBC-solver (not HiGHS) ILP explicitly commented as attacking

"Erdős's Problem 100, the $n = 7$ case" (`source_cbc_origin.py`, released alongside this revision). On closer inspection, however, this specific recovered file has a genuine bug: its C_4-detection logic computes, for each edge (u, v), the common neighbours of u and v – but Q_7 is bipartite, so adjacent vertices never share a common neighbour (a shared neighbour of two adjacent vertices would close an odd cycle), and we confirmed by direct execution that this logic finds 0 of the 672 four-cycles, so the ILP it builds has no C_4 constraints at all. A recovered solver log from the same period (`out.log@@20260227183310`, not bundled) is headed with a Japanese label meaning "fixed version" and explicitly prints, in Japanese, "C4 count: 672 – correctly enumerated", confirming a corrected version of the script existed and was run, reaching 289 edges; but we have not been able to locate that corrected script itself among the recovered files, only its log. We therefore withdraw our earlier characterisation of `source_cbc_origin.py` as the script underlying the 289-edge result: it is a recovered early draft with a confirmed defect, retained in the release for transparency about the development history, but it is not the code that produced 289 edges, and no bit-for-bit reproduction of that specific step is available from what we recovered. The codebase was then revised to use HiGHS with warm-starting, and file timestamps together with recovered solver logs document a progression 289 $\rightarrow$ 301 (`selected_edges_301.json`) $\rightarrow$ 303 (`source_72h.py`'s 72-hour warm-started run, whose log we also recovered and which explicitly records "warm start: loading 301-edge solution" and ends at 303 edges, 0 violations) $\rightarrow$ 304 (via the `sa_search.py` series, dated after the 303-edge result). We further located, in a recovered chat transcript documenting the corrected-CBC run above, that it eventually reached 293 edges (after $\sim$ 12 hours) before switching to HiGHS, which reached 299 edges (in 1 hour, independently confirmed 0 violations at the time), refining the chain to 289 $\rightarrow$ 293 $\rightarrow$ 299 $\rightarrow$ 301 $\rightarrow$ 303 $\rightarrow$ 304. We distinguish two evidentiary levels here: the 293 and 299 figures are read from a historical chat transcript, not independently re-executed by us (we did not re-run a 12-hour CBC solve or re-verify that specific 299-edge output ourselves); the 289 figure is directly documented by a recovered solver log rather than executed by us either. What we *did* independently execute and verify ourselves: the HiGHS-based `source.py` – the released file hardcodes `time_limit=7200` with no command-line override, so we edited this one value down to 60 for this test – run for 60 seconds with no pre-existing `selected_edges_best.json` (a genuine cold start, not a warm start), finds a valid 295-edge C_4-free solution (0 violations) via HiGHS branch-and-bound alone, confirming this ILP approach genuinely produces growing, valid solutions from nothing and is a plausible, demonstrated mechanism for the 301-edge step. We do not have a bit-for-bit reproduction of the exact original 301-edge solution (that would require rerunning HiGHS for the same duration with the same solver-internal randomisation), but the mechanism at each step is now either independently executed by us, or directly documented by a recovered log or chat transcript, rather than merely claimed, closing the previously-unarchived gap about the search method (though not a bit-for-bit replay of any specific historical run). Turning now to `sa_collect304.py`: unlike `phase1_sa`, this collector never needs to escape a far-from-feasible random state: it always starts already at zero violations and only removes a small number of edges before repairing, so the search stays close to feasible throughout. We verified this directly: the released script hardcodes `RUNTIME=36*3600` with no seed and no command-line override, so we edited the runtime down to 30 seconds for this test; running it from its released seed solution then found 1 987 new, valid, previously-unrecorded 304-edge solutions in that one run. As with `sa_q8.py` above, the absence of a fixed seed means this specific count is a one-time observation of the collector's typical throughput, not a number guaranteed to reproduce exactly on a rerun. We further ran a format/implementation-consistency check, independent of our own recovered archive and reproducible from the released files alone: applying `sa_collect304.py`'s own canonicalisation-then-MD5 hash function, run exactly as implemented, directly to each of the 19 866 released `q7_edges_304.jsonl.*` edge sets exactly reproduces the `hash` field already stored in that same record, for all 19 866 with zero mismatches – without needing the recovered `solutions_304.jsonl` at all. We are careful about what this does and does not show: it demonstrates that the released `hash` field is internally consistent with the released `sa_collect304.py` implementation applied to the released edge sets – a statement about format compatibility between two things we release together – not that this script historically generated these edge sets. The historical-provenance evidence for that (the recovered logs, chat transcript, and file timestamps discussed throughout this section) is separate in kind, not strengthened by this hash check; we keep the two distinct rather than treating the format-consistency result as if it were additional provenance evidence. (This 12-hex-digit MD5 `hash` field is unrelated to the full SHA-256 `solution_sha256` column reported by `audit_q7_odd_square.py` and used elsewhere in this paper, e.g. for the global-index-34 witness above; the two are different hash schemes over the same underlying edge sets, not to be confused with each other.) We must correct our own earlier description of what this function does, however: we had described it as coordinate relabelling by descending per-direction edge count, followed by taking the lexicographically smallest of the 2^n bit-flip images. On rereading the code, this describes what a docstring-level intent might have been, not what is implemented. The implementation instead loops over only the n single-bit XOR masks (not all 2^n combined masks), and compares candidate frozensets with Python's `<` operator, which for `frozenset` tests *proper subset*, not lexicographic order; since every candidate here has the same cardinality (304), no candidate is ever a proper subset of another, so this comparison is always `False` and the flip-selection loop never updates its choice. The function's actual output is therefore just the coordinate-permuted set from the first step, with no flip-based canonicalisation occurring at all. We verified this discrepancy directly: recomputing the canonical form as originally (mis)described – trying all $2^n = 128$ XOR masks with a genuine lexicographic comparison – gives a different result from the released implementation for the first published solution (the true lexicographic minimum is attained at XOR mask 107, not reproduced by the actual code), and for 92 of the first 100 released solutions, this correctly-described alternative hash does not match the `hash` field actually stored. There is a second, independent gap in this function's canonicalisation beyond the flip-loop: the coordinate-permutation step sorts directions by descending edge count, but ties are broken only by Python's stable sort (original label order), not by any canonical rule; two graphs in the same $\mathrm{Aut}(Q_7)$-orbit that differ only by permuting two equal-count directions can therefore receive different hashes. We confirmed this directly: the first released solution has direction edge-counts $[44, 43, 44, 42, 43, 44, 44]$ (four tied at 44), hash `79014e2f4aed`; swapping two of the tied 44-edge directions (an automorphism of Q_7) changes the computed hash to `0ba97c019106`. The 19 866/19 866 match we report above is against the code exactly as implemented (with this defect and the flip-loop defect above), which is what we release and what actually produced the stored `hash` values; it is not evidence that full translation-class *or* coordinate-permutation canonicalisation is occurring, and the released 19 866 solutions should not be assumed de-duplicated across the full automorphism group $\mathrm{Aut}(Q_7)$ on this basis (only that they are 19 866 pairwise distinct *labelled* edge sets, as already stated in Proposition 16). We do not know whether the intended, fully-canonicalising version was ever the one actually run in production, or whether this defect was present throughout; we did not attempt to guess or silently correct it, for the same reason given throughout this section. This also fully explains the previously undocumented per-record metadata fields (Section 7 above, as reported in earlier manuscript versions), independently of this defect, since the fields' meanings come from the surrounding collector logic, not from `canonicalize()` itself: `method` records which of the collector's three restart strategies produced that solution (`auto`: random automorphism of a random existing solution; `repair`: direct remove-and-repair of a random existing solution, no automorphism; `auto_seed`: random automorphism of the original seed); the 311 records lacking a `method` field and the 8 carrying a `run` rather than a `trial` field come from an earlier version of the collector script (which we examined during recovery as `sa_collect304 (0).py`, but which we have not included in the released package – it is not among the files a reader receives) that logged records with only a `run` counter and no `method` distinction, before the script was revised to its released form; `trial` is the sequential attempt counter within a single collector run, and `elapsed` the wall-clock seconds since that run started (consistent with a shell history showing the collector was run and re-started under `nohup` multiple times, explaining why `trial` resets are visible across the released file). We also note a genuine implementation defect in the recovered `sa_search.py` itself: `cur_list`/`mis_list` (the candidate lists sampled for removal/swap moves) are rebuilt from `current_set`/`current_missing` only every `REBUILD_INTERVAL`$=$ 5 000 iterations, while ordinary add/remove moves update `current_set`/`current_missing` directly every iteration without refreshing these lists; a stale list entry can therefore be chosen for removal or swap, and since `delta_remove` does not check that its argument edge is actually present, the tracked `current_v` can in principle drift from the true violation count. `count_violations` is called once, to initialise `current_v` at the start of each run, but is not called again to re-verify before `save_best` writes an improved solution to disk. This is a defect in the search code's internal self-consistency, not in the released certificates: every released 304-edge solution – regardless of which script produced it – has been independently re-verified C_4-free by exhaustive enumeration in this manuscript (Section 2), so the defect does not affect the correctness of $\mathrm{ex}(Q_7, C_4) \geq 304$. We release the code with this defect documented, not silently patched, for the same reason given throughout this section: patching it would misrepresent what was actually run. We release `sa_search.py`, `sa_collect304.py`, and the seed solution alongside this revision as recovered code for the Q_7 results, with the two caveats above (the seed's provenance below 304 edges, and this list-staleness defect); see Section 7.2 below for the separate, distinct `c4free_sa.py` whose relationship to the Q_8 results (and to the Q_7 results, before this recovery) remains as previously described. The 1 076-trial statistic at 305 edges and the analogous 681-edge search-attempt statistic for Q_8 are unaffected by this recovery and remain unarchived, unresolved provenance claims in the sense described below; only the 680-edge *solutions* themselves (not the 681-edge negative-search statistics) have since also been traced to verified code, described next.

7.1 Update: the Q_8 680-edge production code has also been recovered

We have since also located and verified the actual production code for the two released Q_8 680-edge solutions: `sa_q8.py`, a penalty-SA hill-climber structurally similar to `c4free_sa.py`'s Phase-1/Phase-2 design (and sharing its per-trial hard violation cap, `max_viol`, drawn from $[8, 35]$) but critically different in one respect: each trial restarts not from a fresh random sample but from `best_valid_sol`, the current best *zero-violation* solution (initially an edge-greedy construction, then whatever the algorithm has since improved upon), loaded from `q8_best.json`. Because every restart point already has zero violations, the freezing failure mode of `c4free_sa.py` does not arise: proposals only need to stay within `max_viol` of an already-feasible state, not close an initial gap of 100+ violations. The recovered directory contains a progression of intermediate saved solutions (584, 662, 666, 669, 670, 672–680 edges), consistent with gradual hill-climbing improvement over many trials. We verified two things directly: first, that the recovered `q8_edges_680.txt` and `q8_best.json` match the released Solution A and Solution B (Section 4) exactly by edge set (we release these two files as `q8_solution_a.txt`/`q8_solution_b.json` alongside this revision, so a reader can repeat this edge-set comparison directly); second, by direct execution – resuming from the 670-edge intermediate state under an external 90-second wall-clock cutoff (`timeout 90 python3 sa_q8.py`, not the script's own `RUNTIME` variable) – that the code makes genuine progress, reducing the violation count at the 675-edge target from 22 to 8 within that window. We release this 670-edge checkpoint as `q8_checkpoint_670.json`; to repeat this second test, copy it to `q8_best.json` in the same directory as `sa_q8.py` before running (the script only autoloads a file named exactly `q8_best.json`), and use an external timeout as above rather than editing the internal `RUNTIME` variable: `RUNTIME` is checked only once per outer trial, while each trial's inner Phase-1 loop runs a fixed 2–12 million steps with no time check inside it, so setting `RUNTIME` to a small value does not reliably stop execution near that value – only an external, process-level timeout does. `sa_q8.py` sets no random seed, so a rerun will show the same qualitative monotonic-improvement behaviour (confirming the code is not frozen, unlike `phase1_sa`) but not necessarily the same specific violation-count trajectory or the exact 22 $\to$ 8 figures – this is a one-time observation of genuine progress, not a fixed-seed reproducible numeric result; without this file, `sa_q8.py` instead starts from a fresh greedy construction, which is a different (and also legitimate, since the algorithm always restarts from a valid state either way) starting point than the one we tested from. We also release `sa_q8.py` itself. The 1 076-trial statistic at 681 edges (Section 4) is a record of attempts to exceed 680, not of producing the 680-edge solutions themselves; we did not recover a script or log specifically substantiating that count, so it remains an unarchived, author-recorded claim in the same sense as before.

7.2 The released `c4free_sa.py` and its known defect

We have since determined why `c4free_sa.py` does not match the production history: it was written after the fact. Following an early external review (of the first arXiv submission, which did not yet include search source code) that flagged the absence of published search code as the submission's principal weakness, `c4free_sa.py` was written on 2026-03-28 – after the Q_7 (19 866 solutions, complete by 2026-03-17) and Q_8 (680-edge solutions, complete by 2026-03-21) results already existed via the separate scripts recovered and described above – specifically to have *some* search code available for release, and was validated only on the much smaller Q_5 case (56 edges) at the time, not at the Q_7/Q_8 scale where the `viol_cap` defect documented below becomes fatal. This explains, rather than excuses, the discrepancy: `c4free_sa.py` was never the operative research code for either dimension; it was a retrospective, small-scale-validated addition made to satisfy a reviewer's reproducibility request, and the defect went undetected because it was not exercised at the scale that triggers it. The quantitative claim about the Q_8 *search process*

— "1 076 independent trials" at 681 edges — remains an unarchived record, as noted just above; before the two recoveries above, this described the Q_7 and the Q_8 680-edge solution claims as well. The search program `c4free_sa.py`, distinct from both recovered production-code pairs above, is released alongside this manuscript as originally reported, but the specific run seeds, logs, and driver invocations that would substantiate the 681-edge, 1 076-trial statistic are not supplied, and we no longer believe `c4free_sa.py` as released is what produced either the Q_7 or the Q_8 solutions (see the recoveries above for what we now believe did). This process-level claim should be read as the authors' record of the search that was run, not as an independently reproducible certificate in the same sense as the edge sets. Worse, as detailed below, we found on direct execution that the released, unmodified `c4free_sa.py` frequently cannot progress at all from a fresh random start under the documented parameter ranges, for a structural reason (not merely slow convergence); we could not identify parameters under which it reaches the target edge count with zero violations. A reader wishing to verify the exact 681-edge process-level statistics should therefore not assume rerunning `c4free_sa.py` as documented will reproduce them, or indeed complete a single trial successfully; see below for the specifics.

Beyond the aggregate counts, each released Q_7 record itself carries the provenance fields (`method`, `run`, `trial`, `hash`, `elapsed`) explained above. Of the 19 866 records, `method` is `auto` for 18 206, `auto_seed` for 1 347, `repair` for 2, and absent for 311; a `run` field is present on only 8 records (one with `run=0`, seven with `run=2`); and 19 858 records carry a `trial` field with values up to at least seven figures. These fields are now explained by the recovered `sa_search.py/sa_collect304.py` pipeline above, subject to the two caveats noted there (the seed's own provenance below 304 edges, and the list-staleness defect); this record is substantially more resolved than the analogous Q_8 681-edge trial statistic, which remains an unarchived claim with no recovered script or log at all (the Q_8 680-edge *solutions* themselves, by contrast, have also now been traced to recovered code – see below).

7.3 Two-phase simulated annealing

Phase 1 (Penalty SA). The acceptance criterion for proposed moves minimises $f(E) = -|E| + \lambda V$ where V is the number of C_4 violations and $\lambda \in [0.30, 0.90]$, but the "best" state `phase1_sa` tracks and returns is *not* the minimiser of f: the released code instead keeps the state with lexicographically smallest $(V, -|E|)$ seen so far (`v_cur < best_v or (v_cur == best_v and size_cur > best_size)`) – prioritising fewer violations first, and only using edge count as a tiebreaker among states with equal V. Temperature schedule: geometric from $T_0 \in [0.20, 4.00]$ to $T_1 \in [0.001, 0.030]$ over 3–15 million steps. A move toggles a single edge; the change in f is computed incrementally in $O(\deg)$ time using precomputed C_4-to-edge incidence lists.

Phase 2 (Swap SA). Edge count is fixed; swap moves (remove one edge, add one non-edge) minimise V. Phase 2 only runs when phase 1 *returns* a best-so-far state with $V \leq 4$ violations – `phase1_sa` tracks and returns the best state seen during the run, not necessarily its terminal current state, so a literal reimplementation using only the terminal state could behave differently. In the released code, the step budget for phase 2 is chosen by the phase-1 violation count v, not by dimension: 2×10^8 steps if $v \leq 1$, otherwise 5×10^7 steps. In practice this correlates with dimension, since phase 1 typically ends closer to $v \leq 1$ at the Q_7/305-edge target than at the Q_8/681-edge target, but the branch itself is on v, not on n or the target edge count.

Diversification. The released search code (`c4free_sa.py`) computes, before each trial, a random element of $\mathrm{Aut}(Q_n)$ (random bit permutation composed with random bit flip) applied to the current best solution, intended as a diversified restart point. However, this computed value is not passed into `phase1_sa`, which unconditionally initialises from a fresh uniform-random edge set of the target size (`rng.sample(range(ne), target)`) regardless of any prior best solution; the automorphism-based restart is dead code in the released implementation. (The script's own docstring originally described this mechanism without noting the bug; we have appended a note to the docstring documenting it, without altering any functional code, so that a reader of the code alone is not misled.)

A more serious defect in `phase1_sa`. On direct execution, we found that the released `phase1_sa` is frequently unable to make *any* progress at all, for a structural reason independent of step count. Each accepted or rejected proposal toggles a single edge, changing the violation count V by at most $n-1$ (the number of squares containing that edge); a proposal is rejected outright whenever the resulting V would exceed `viol_cap` (drawn per trial from $[6, 40]$). If the initial random edge set's V exceeds `viol_cap` $+(n-1)$, *no* single-edge toggle can ever produce an accepted move, so the state is frozen from step one, for any number of steps. We verified this is not a rare occurrence but the typical case: reproducing the exact RNG state of trial 1 for `--n 7 --target 304 --seed 42`, the initial violation count is $V = 152$ against `viol_cap` $= 14$ (best single-toggle result: 147, still far above cap), and running `phase1_sa` for 100 000 steps leaves $V = 152$ unchanged. Reproducing trial 1 of `--n 8 --target 680 --seed 0` similarly gives initial $V = 333$ against `viol_cap` $= 38$, frozen after 50 000 steps. Sampling 10 000 independent uniform-random edge sets directly (bypassing the annealing loop; `random.Random(999)`, `rng.sample(range(ne), target)` repeated 10 000 times per dimension, minimum V tracked via `count_violations`) gives a minimum V of 117 for $Q_7/304$ and 297 for $Q_8/680$ – both far above the maximum possible `viol_cap` of 40 – so this is not attributable to unlucky seeds; independent resampling with other seeds gives minima in the same range (we observed 111–117 for Q_7 and 296–304 for Q_8 across several seeds), so the qualitative conclusion (minimum $V \gg 40$) does not depend on the particular seed chosen, though the exact minimum does. We could not identify parameter draws under which the released, unmodified `phase1_sa` reaches $V = 0$ from a fresh random start for these targets. Consequently, our earlier claim that fresh per-trial random initialisation explains the diversity of the released solutions should be read with this caveat: we can confirm the released *solutions* are valid (independently verified C_4-free certificates), but the released, unmodified `phase1_sa` – run as literally documented – cannot be what produced them, for the structural reason just demonstrated (not merely bad luck with parameters). We now know why: as detailed in Section 7.2 above, `c4free_sa.py` was itself written after the fact (dated 2026-03-28), once the Q_7 and Q_8 results already existed via the separately-recovered scripts described above, and validated only at the much smaller Q_5 scale at the time – it was never a candidate for the historical production code in the first place, so there is no remaining question of whether some undocumented earlier version of *this script* matches a different historical parameter setting. The genuine remaining gaps are narrower and specific: the very first sub-304-edge bootstrap step of `sa_search.py`'s own lineage (Section 7 above), and the unarchived 681-edge/1 076-trial statistic (Section 4), neither of which this section's finding about `phase1_sa` bears on directly.

The diversity actually observed across the 19 866 collected solutions is therefore not fully explained by the mechanisms described in this section; we note for completeness that even had the intended automorphism-based restart mechanism been wired correctly, applying an automorphism to a solution does not change which $\mathrm{Aut}(Q_n)$-orbit it belongs to, and the annealing moves used here (uniform edge toggle/swap proposals against an automorphism-invariant objective) admit an $\mathrm{Aut}(Q_n)$-equivariant Markov kernel; any diversification benefit would have come only from decorrelating the specific labelled trajectory taken by a finite stochastic run; it does not diversify the orbit sampled from.

7.4 Comparison with known lower bounds

Brass–Harborth–Nienborg's weaker bound is stated for $n \geq 9$; the values below evaluate that formula at $n = 7, 8$ as an extrapolation for illustrative comparison, not as a proven bound at those dimensions (see Table 1 and the Remark in Section 4).

$$n = 7: \quad \tfrac{1}{2}(7 + 0.9\sqrt{7}) \cdot 64 \approx 300.2 \text{ (extrapolated)}, \quad \text{bound: } 304\ (\Delta = +3.8,\ +1.27\%),$$
$$n = 8: \quad \tfrac{1}{2}(8 + 0.9\sqrt{8}) \cdot 128 \approx 674.9 \text{ (extrapolated)}, \quad \text{bound: } 682\ (\Delta = +7.1,\ +1.05\%).$$

The $n = 8$ figure is the odd-square reconstruction of Section 5; the simulated-annealing method of this section independently reaches 680 edges on Q_8 ($\Delta = +5.1$, $+0.75\%$; Section 4).

7.5 Regularity trend across dimensions

Degree sequences across dimensions: Q_6: $\{4^{56}, 5^{8}\}$; Q_7: $\{4^{32}, 5^{96}\}$; Q_8: $\{4^{8}, 5^{160}, 6^{88}\}$ (Solution A; Section 4). The minimum degree is 4 in all three cases; the degree support is $\{4, 5\}$ for both Q_6 and Q_7 (unchanged from $n = 6$ to $n = 7$) and widens to $\{4, 5, 6\}$ only at Q_8. Beyond this, the exponents do not follow an evident closed-form pattern: Q_6's $\{56, 8\}$ split is markedly more skewed than Q_7's $\{32, 96\}$ or Q_8's $\{8, 160, 88\}$, so we do not claim a general regularity trend here.

8 Computational Reproduction of the $\mathrm{ex}(Q_6, C_4) = 132$ Lower Bound

Harborth and Nienborg [9] proved $\mathrm{ex}(Q_6, C_4) = 132$ by combinatorial arguments. We reproduce the lower-bound (construction) side fully independently and computationally (Section 8.1); for the upper-bound side we give an ILP formulation whose feasible value matches 132 (Section 8.2), but we did not independently verify its optimality within a practical runtime, so the upper bound $\mathrm{ex}(Q_6, C_4) \leq 132$ itself rests on [9], not on our own solve.

8.1 Lower bound: explicit construction

Simulated annealing (Section 7) is reported to have found a C_4-free subgraph of Q_6 on 132 edges, certified by exhaustive enumeration of all 240 four-cycles. The explicit edge list is provided in `q6_edges_132.jsonl`. As with the Q_7/Q_8 constructions (Section 7), the released, unmodified `phase1_sa` exhibits the same structural freezing at $n = 6$, target 132: reproducing trial 1 for `--n 6 --target 132 --seed 42` gives initial $V = 54$ against `viol_cap` $= 14$ (best single-toggle result 49, still above cap), frozen after 100 000 steps; `--seed 0` gives initial $V = 52$ against `viol_cap` $= 38$ (best single-toggle result 48). The 132-edge certificate itself remains independently verified as a valid C_4-free construction;

as for Q_7, Q_8, we cannot confirm the released code, run as documented, is what actually produced it.

8.2 Upper bound: integer linear program

Label the 192 edges of Q_6 as $x_0, \ldots, x_{191} \in \{0, 1\}$ (matching the released `q6_ilp.mps`'s 0-indexed variable names; the text elsewhere in this paper is otherwise 1-indexed for readability, but the MPS file itself uses 0-indexing throughout), and let $\mathcal{C}_6$ denote the set of all 240 four-cycles of Q_6. For each four-cycle $\{e_1, e_2, e_3, e_4\} \in \mathcal{C}_6$, the constraint $x_{e_1} + x_{e_2} + x_{e_3} + x_{e_4} \leq 3$ ensures C_4-freeness. The ILP is:

$$\max \sum_{i=0}^{191} x_i \quad \text{subject to} \quad \sum_{e \in C} x_e \leq 3 \;\; \forall C \in \mathcal{C}_6, \quad x_i \in \{0, 1\}.$$

This formulation has 192 binary variables and 240 constraints, each variable appearing in exactly 5 of the 240 square constraints (matching Q_6's edge-square incidence). The MPS file `q6_ilp.mps` is provided for independent verification (SHA-256 in the released checksum manifest). The correspondence between each x_i and its Q_6 edge was not originally recorded alongside the MPS file. We have since constructed an incidence-preserving identification (`q6_ilp_edge_map.csv`: $x_i \leftrightarrow$ vertex pair, verified to reproduce all 240 constraints' variable memberships exactly from the released MPS file) by matching the MPS's variable–constraint incidence pattern against Q_6's known edge–square incidence structure under one specific, standard enumeration convention (integer vertex labels $0, \ldots, 63$; squares enumerated by base vertex, then direction pair). We call this an identification rather than *the* original correspondence because incidence structure alone determines it only up to $\mathrm{Aut}(Q_6)$: any automorphism compatible with the assumed labelling convention would give an equally valid, incidence-matching relabelling. The identification is nonetheless useful for interpreting individual solution vectors and is consistent with (not merely coincidentally matching) the released data throughout.

Proposition 18. *The ILP above has feasible value* 132*, matching the released 132-edge construction (Section 8.1); combined with Harborth and Nienborg's independent combinatorial proof that* $\mathrm{ex}(Q_6, C_4) \leq 132$ *[9], this gives* $\mathrm{ex}(Q_6, C_4) = 132$*. We did not independently verify optimality of* 132 *for this ILP instance within a practical runtime (see below); the upper bound* $\mathrm{ex}(Q_6, C_4) \leq 132$ *used here rests on [9], not on our own solve of the ILP.*

We independently re-solved `q6_ilp.mps` with HiGHS 1.15.1 (Python bindings via `highspy`; default settings, single thread, single-threaded x86_64 Linux VM, Intel Xeon @ 2.10GHz, 4GB RAM). A feasible solution matching the released 132-edge construction was found quickly, but the solver did not close the optimality gap within several minutes under these default settings (a several-percent gap, dual bound not yet at the -132 primal value). As this figure is implementation- and machine-dependent, it should be read as evidence that the gap does not close quickly under a generic default-settings solve, not as a precise benchmark. This is consistent with the large automorphism group of Q_6 ($|\mathrm{Aut}(Q_6)| = 6! \cdot 2^6 = 46\,080$) producing many symmetric alternative optima, which is known to slow generic branch-and-bound without dedicated symmetry-breaking. We do not claim generic MIP solvers verify this instance's optimality quickly; the 132-edge value should be regarded as independently established by Harborth–Nienborg's combinatorial proof [9], with the ILP providing a machine-checkable feasible witness for the primal side and a formulation that a sufficiently long or specialised solve can, in principle, verify on the dual side.

9 Open Problems

1. Prove or disprove $\mathrm{ex}(Q_7, C_4) = 304$ (Conjecture 17). A direct SAT-encoding attack on this question (a 305-edge C_4-free existence query encoded as CNF, 64 512 variables and 129 104 clauses, with a symmetry-broken variant at 113 664 variables/227 786 clauses; attempted cube-and-conquer splitting via `march_cu` and direct solving via CaDiCaL and Kissat) was attempted over several weeks and did not reach a conclusion: the cube-generation step itself repeatedly failed to complete, and direct solver runs of over 17 days of process time returned `UNKNOWN` rather than a SAT/UNSAT verdict. A second, independent attempt using a different encoding (Sinz's sequential at-most-143 cardinality encoding on the negated edge variables, arriving at the same 64 512-variable, 129 104-clause scale by a different construction) and solving directly with Kissat (no cube splitting) was run for over 408 hours (17 days) of wall-clock time and likewise terminated with no SAT/UNSAT verdict logged. A third, independent approach – direct branch-and-bound on the original 448-variable ILP with added symmetry-breaking constraints (685 rows total), using HiGHS and warm-started from the known 304-edge solution – ran for over 365 000 seconds (~ 101 hours) without closing the optimality gap, which remained at 5.26% (best bound ≈ -320.0 against the incumbent -304) when the log ends. We record all three as unsuccessful attempts, not as evidence either way; the hardness suggests this instance is not easily settled by off-the-shelf SAT or MIP solvers at current scale.

2. Determine $\mathrm{ex}(Q_8, C_4)$ exactly. Is it 682, or can a non-odd-square construction exceed $g(8) = 682$? The Q_7 audit of Section 6.4 shows the two questions are genuinely different for $n = 7$ (the general extremal value 304 is attained overwhelmingly by non-odd-square solutions), so no assumption is made here about which way $n = 8$ resolves.

3. What is the number of $\mathrm{Aut}(Q_7)$-orbits among all 304-edge C_4-free subgraphs of Q_7? Do the 19 866 released solutions represent every such orbit? Among the 389 odd-square solutions specifically?

4. Is the degree sequence $\{4^{32}, 5^{96}\}$ necessary for any locally maximal C_4-free subgraph of Q_7 on 304 edges?

5. Can the flag algebra method of [1, 2] yield tight bounds for small $n \leq 8$?

6. Does the degree support of locally maximal C_4-free subgraphs of Q_n near the extremal edge count widen beyond $\{4, 5\}$ again for $n \geq 9$ (as it did once, from Q_7 to Q_8; Section 7.5), and is minimum degree 4 necessary at these edge counts for $n \geq 9$?

7. We now know $g(6) = \mathrm{ex}(Q_6, C_4) = 132$ (Theorem 2 combined with [9]). For $n = 7, 8$, Theorem 2 only establishes $g(7) = 304$ and $g(8) = 682$; whether $\mathrm{ex}(Q_7, C_4) = g(7)$ and $\mathrm{ex}(Q_8, C_4) = g(8)$ remains exactly as open as Conjecture 17 and Open Problem 2 above, since $\mathrm{ex}(Q_n, C_4)$ itself is not known exactly for $n = 7, 8$. (The coincidence at $n = 6$ is in any case not explained by the odd-square construction alone for

$n = 7$: only 389 of the 19 866 known 304-edge Q_7 solutions are odd-square.) Does $g(n) = \mathrm{ex}(Q_n, C_4)$ continue to hold for $n = 9, 10, \ldots$ (once $\mathrm{ex}(Q_n, C_4)$ is itself determined), or does the gap $\mathrm{ex}(Q_n, C_4) - g(n)$ become positive at some n? Laplante et al. [10] give general lower and upper bounds on the frustration index $|E(Q_n)| - g(n)$ that match only when n is a power of 4; whether $g(n) = \mathrm{ex}(Q_n, C_4)$ persists beyond $n = 8$ is, in their language, a question about when this frustration-index gap and the C_4-free extremal gap coincide.

Acknowledgements

The author thanks B. Lidický (Iowa State University) for the endorsement and for pointing out the reference [1]. The author thanks S. Lai for identifying the relevance of [5, 11] to the Q_7 and Q_8 constructions of this paper and for independently checking the reconstructed odd-square material of Section 5. The author also thanks the open-source community for the HiGHS and SCIP solvers used in ILP experiments.

Use of generative AI. The author used Claude (Anthropic) for: (i) language editing of the abstract; (ii) assistance in writing the supplementary verification script (`verify.py`) used to confirm C_4-freeness of the released edge-list certificates; and, for this revision, (iii) retrieval and cross-checking of the primary sources cited in Section 5 and (iv) drafting assistance for the revised text. All definitions, constructions, theorems, computations, and analyses are the author's own, including the 256-bit spin configuration used in the Q_8 witness of Section 5.3, whose provenance is recorded there as a dated correspondence account. The author reviewed all AI-assisted output, independently executed the verification and witness scripts against the released data, and takes full responsibility for the accuracy and integrity of the work. The specific model version(s) of Claude used across the original submission and this revision were not recorded at the time and cannot be confirmed retrospectively; the disclosure above is accurate as to which tasks AI assistance was used for, but not as to the exact model version for each.

Disclosure statement

No potential conflict of interest was reported by the author.

References

[1] R. Baber, Turán densities of hypercubes, arXiv:1201.3587, 2012.

[2] J. Balogh, P. Hu, B. Lidický, and H. Liu, Upper bounds on the size of 4- and 6-cycle-free subgraphs of the hypercube, *Eur. J. Combin.* **35** (2014), 75–85.

[3] P. Brass, H. Harborth, and H. Nienborg, On the maximum number of edges in a C_4-free subgraph of Q_n, *J. Graph Theory* **19** (1995), 17–23.

[4] F. Chung, Subgraphs of a hypercube containing no small even cycles, *J. Graph Theory* **16** (1992), 273–286.

[5] B. Derrida, Y. Pomeau, G. Toulouse, and J. Vannimenus, Fully frustrated simple cubic lattices and the overblocking effect, *J. Physique* **40** (1979), 617–626.

[6] M. R. Emamy-K., P. Guan, and P. I. Rivera-Vega, On the characterization of the maximum squareless subgraphs of the 5-cube, *Congr. Numer.* **88** (1992), 97–109.

[7] F. Harary, On the notion of balance of a signed graph, *Michigan Math. J.* **2** (1953), 143–146.

[8] P. Erdős, On some problems in graph theory, combinatorial analysis and combinatorial number theory, in: *Graph Theory and Combinatorics* (B. Bollobás, ed.), Academic Press, 1984, pp. 1–17.

[9] H. Harborth and H. Nienborg, Maximum number of edges in a six-cube without four-cycles, *Bull. ICA* **12** (1994), 55–60.

[10] S. Laplante, R. Naserasr, A. Sunny, and Z. Wang, Sensitivity conjecture and signed hypercubes, *ACM Trans. Comput. Theory* **18** (2026), no. 1, Article 6, 1–25, https://doi.org/10.1145/3777401 (preprint: HAL hal-04080411, 2023).

[11] E. Marinari, G. Parisi, and F. Ritort, The fully frustrated hypercubic model is glassy and aging at large D, *J. Phys. A* **28** (1995), 327–334, arXiv:cond-mat/9410089.

[12] A. Thomason and P. Wagner, Bounding the size of square-free subgraphs of the hypercube, *Discrete Math.* **309** (2009), 1730–1735.